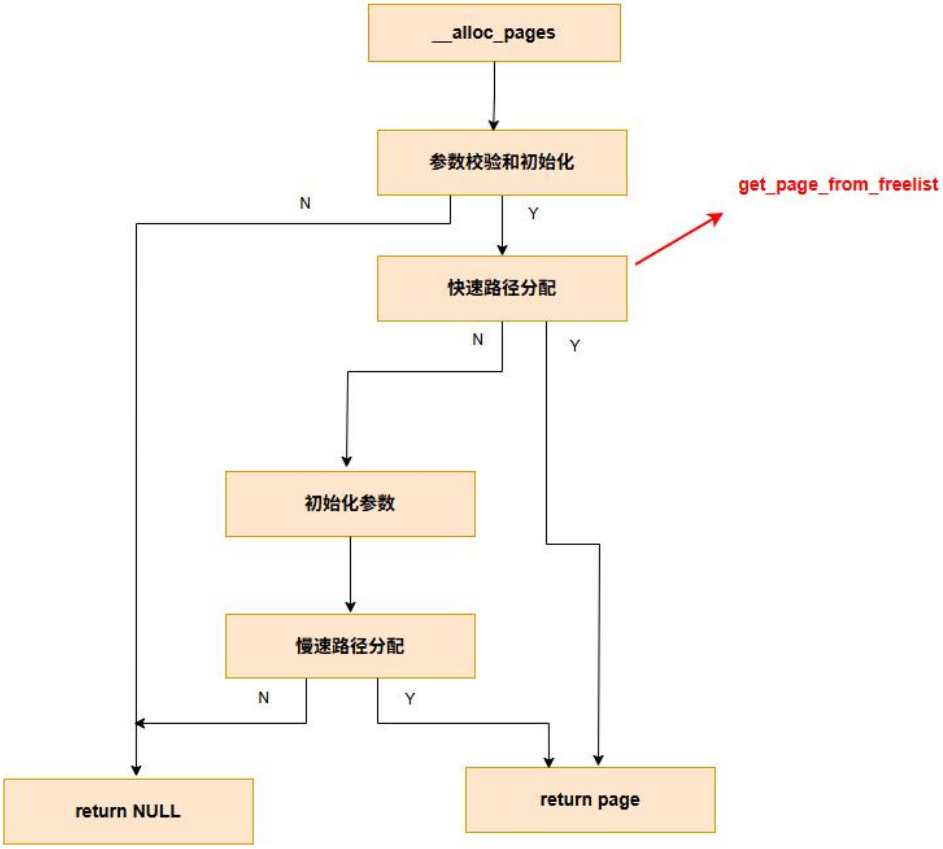
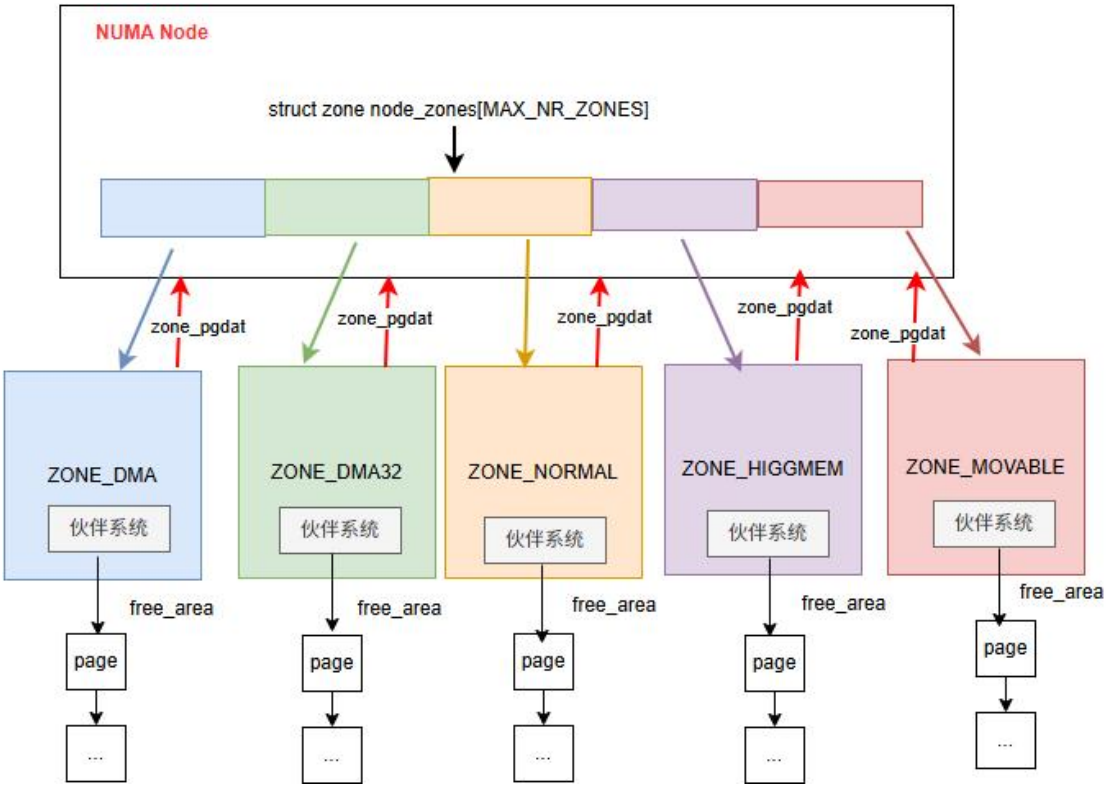
6 Linux 伙伴系统的设计与实现 2
6.1 伙伴系统的核心数据结构 3
6.2 伙伴系统的内存分配原理 7
6.3 伙伴系统的内存回收原理 13
6.4 伙伴系统的内核源码分析 14
6.4.1 get_page_from_freelist 函数分析 14
6.4.2 获取内存区域 zone 里指定的内存水位线 20
6.4.3 检查 zone 中剩余内存容量是否满足水位线要求 21
6.4.4 伙伴系统的实现 26
6.4.5 从 CPU 高速缓存列表中获取内存页 28
6.4.6 从伙伴系统中获取内存页 30
6.4.6.1 __rmqueue_smallest 伙伴系统的核心实现 33
6.4.6.2 __rmqueue 伙伴系统的 fallback 实现 35
6.4.7 内存分配成功之后初始化 page 42
6.5 内存释放源码实现 44
6.5.1 将内存释放到 CPU 高速缓存列表 pcplist 中 45
6.5.2 将内存释放到伙伴系统中 50
6.5.3 free_one_page 函数源码分析 51

6 Linux 伙伴系统的设计与实现

伙伴系统内存分配逻辑整体流程图：



6.1 伙伴系统的核心数据结构



在内核中的物理内存管理架构中。内核会为 NUMA 节点中的每个物理内存区域 zone 分配一个伙伴系统用于管理该物理内存区域 zone 里的空闲内存页。伴系统的核心数据结构也封装在 struct zone 里面。

```
struct zone {

    ......省略......

    // 伙伴系统所管理的物理内存页个数
    atomic_long_t       managed_pages;

    ......省略......

    // 伙伴系统的核心数据结构
    struct free_area    free_area[NR_PAGE_ORDERS];

    ......省略......

}
```

managed_pages 用于表示该内存区域内被伙伴系统所管理的物理内存页数量。计算方式是：通过 present_pages （不包含内存空洞）减去内核为应对紧急情况而预留的物理内存页 reserved_pages 得到的。所以，伙伴系统所管理的空闲物理内存页并不包含紧急预留内

存。

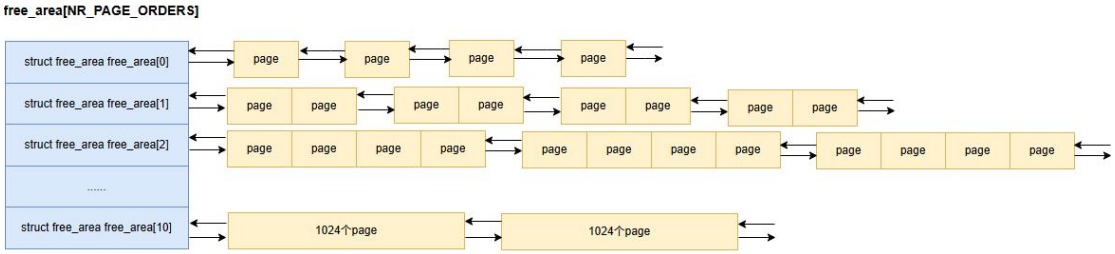
伙伴系统所分配的物理内存页全部都是物理上连续的，并且只能分配 2 的整数幂个页，这里的整数幂在内核中称之为分配阶 order。调用物理内存分配接口时，均需要指定这个分配阶 order，意思是从伙伴系统申请多少个物理内存页，假设我们指定分配阶为 order，那么就会从伙伴系统中申请 2 的 order 次幂个物理内存页。

伙伴系统会将物理内存区域中的空闲内存根据分配阶 order 划分出不同尺寸的内存块，并将这些不同尺寸的内存块分别用一个双向链表组织起来。比如：分配阶 order 为 0 时，对应的内存块就是一个 page。分配阶 order 为 1 时，对应的内存块就是 2 个 pages。依次类推，当分配阶 order 为 n 时，对应的内存块就是 2 的 order 次幂个 pages。

NR_PAGE_ORDERS - 1 是内核中规定的分配阶 order 的最大值，定义在 **include\linux\mmzone.h** 中最大分配阶 NR_PAGE_ORDERS - 1 = 10，也就是说一次，最多只能从伙伴系统中申请 1024 个内存页，对应 4M 大小的连续物理内存。

```
#ifndef CONFIG_ARCH_FORCE_MAX_ORDER
#define MAX_PAGE_ORDER 10
#else
#define MAX_PAGE_ORDER CONFIG_ARCH_FORCE_MAX_ORDER
#endif

#define NR_PAGE_ORDERS (MAX_PAGE_ORDER + 1)
```

数组 free_area[NR_PAGE_ORDERS] 中的索引表示的就是分配阶 order，用于指定对应双向链表组织管理的内存块包含多少个 page。



struct free_area 主要描述的就是相同尺寸的内存块在伙伴系统中的组织结构，nr_free 则表示的是该尺寸的内存块在当前伙伴系统中的个数，这个值会随着内存的分配而减少，随着内存的回收而增加。

```
struct free_area {
    struct list_head    free_list[MIGRATE_TYPES];
    unsigned long       nr_free;//该尺寸的内存块在当前伙伴系统中的个数
};

struct list_head {
    // 双向链表
    struct list_head *next, *prev;
};
```

nr_free 表示的可不是空闲内存页 page 的个数，而是空闲内存块的个数，对于 0 阶的内存块来说 nr_free 确实表示的是单个内存页 page 的个数，因为 0 阶内存块是由一个

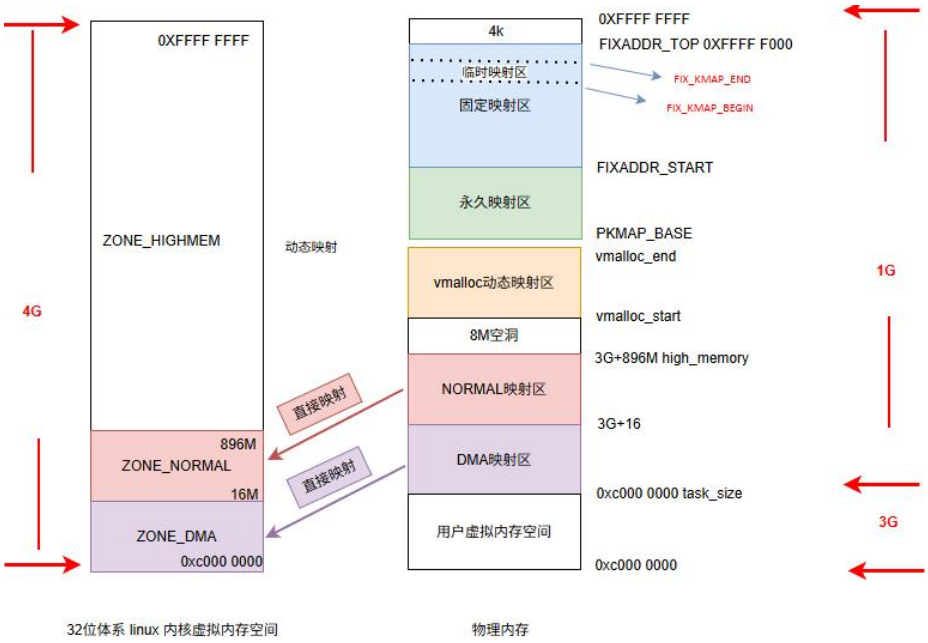
page 组成的，但是对于 1 阶内存块来说，nr_free 则表示的是 2 个 page 集合的个数，以此类推对于 n 阶内存块来说，nr_free 表示的是 2 的 n 次方 page 集合的个数。

这些相同尺寸的内存块在 struct free_area 结构中通过 struct list_head 结构类型的双向链表统一组织起来。

内核使用多个双向链表来组织这些相同尺寸的内存块，这些双向链表组成一个数组 free_list[MIGRATE_TYPES]，该数组中双向链表的个数为 MIGRATE_TYPES。 MIGRATE_TYPES 称为物理内存页的迁移类型。内核根据物理内存页的迁移类型将这些相同尺寸的内存块近一步通过不同的双向链表重新组织起来。也就是说，free_area 是将相同尺寸的内存块组织起来，free_list 是在 free_area 的基础上近一步根据页面的迁移类型将这些相同尺寸的内存块划分到不同的双向链表中管理。

物理内存页面的迁移类型 MIGRATE_TYPES 定义在 **include\linux\mmzone.h** 中，

```
enum migratetype {
    MIGRATE_UNMOVABLE,// 不可移动
    MIGRATE_MOVABLE, // 可移动
    MIGRATE_RECLAIMABLE,// 可回收
    MIGRATE_PCPTYPES,  // 属于 CPU 高速缓存中的类型，
                        PCP 是 per_cpu_pageset 的缩写
    MIGRATE_HIGHATOMIC = MIGRATE_PCPTYPES, // 紧急内存
#ifdef CONFIG_CMA
    MIGRATE_CMA, // 预留的连续内存 CMA
#endif
#ifdef CONFIG_MEMORY_ISOLATION
    MIGRATE_ISOLATE,    /* can't allocate from here */
#endif
    MIGRATE_TYPES //不代表任何区域，只是单纯表示一共有多少个迁移类型
};
```



MIGRATE_UNMOVABLE 表示不可移动的页面类型，这种类型的物理内存页面是固定的不能随意移动，内核所需要的核心内存大多数是从 MIGRATE_UNMOVABLE 类型的页面中进行分配，这部分内存一般位于内核虚拟地址空间中的**直接映射区**。

在内核虚拟地址空间的直接映射区中，虚拟内存地址与物理内存地址都是直接映射的，虚拟内存地址通过减去一个固定的偏移量就可以直接得到物理内存地址，由于这种直接映射的关系，所以这部分内存是不能移动的，因为一旦移动虚拟内存地址就会发生变化，这样一来虚拟内存地址减去固定的偏移得到的物理内存地址就不一样了。

MIGRATE_MOVABLE 表示可以移动的内存页类型，这种页面类型一般用于在进程用户空间中分配，因为在用户空间中虚拟内存与物理内存都是通过页表来动态映射的，物理页移动之后，只需要改变页表中的映射关系即可，而虚拟内存地址并不需要改变。一切对进程来说都是透明的。

MIGRATE_RECLAIMABLE 表示不能移动，但是可以直接回收的页面类型，比如前面提到的文件缓存页，它们就可以直接被回收掉，当再次需要的时候可以从磁盘中继续读取生成。或者一些生命周期比较短的内存页，比如 DMA 缓存区中的内存页也是可以被直接回收掉。

MIGRATE_PCPTYPES 则表示 CPU 高速缓存中的页面类型，PCP 是 per_cpu_pageset 的缩写，每个 CPU 对应一个 per_cpu_pageset 结构，里面包含了高速缓存中的冷页和热页。

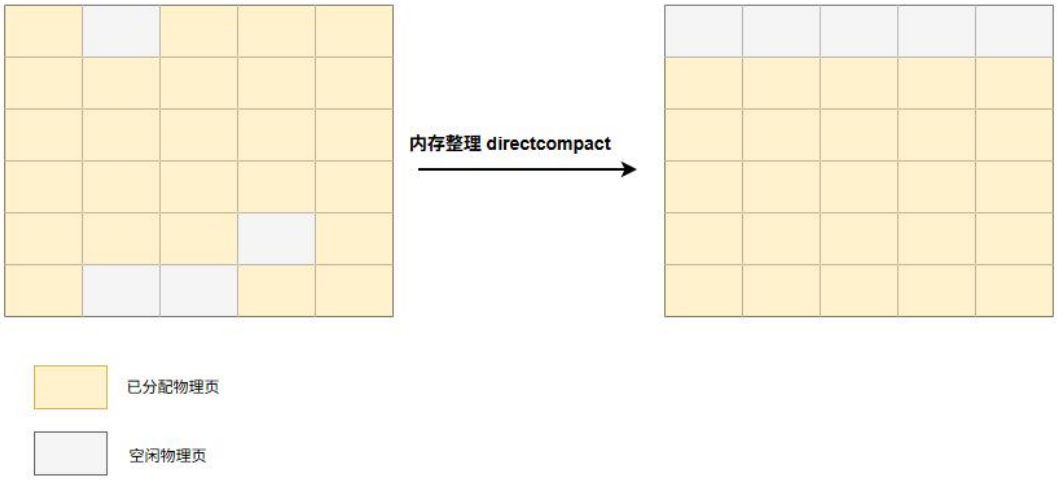
MIGRATE_CMA 表示属于 CMA 区域中的内存页类型，CMA 的全称是 contiguous memory allocator，顾名思义它是一个分配连续物理内存页面的分配器用于分配连续的物理内存。

内核在系统刚刚启动时，先预留一部分连续的物理内存，这部分物理内存就是 CMA 区域，这部分内存可以被进程正常的使用，当有连续内存分配需求时，内核会通过页面回收或者迁移的方式将这部分内存腾出来给 CMA 分配。CMA 的初始化是在伙伴系统初始化之前完成的。

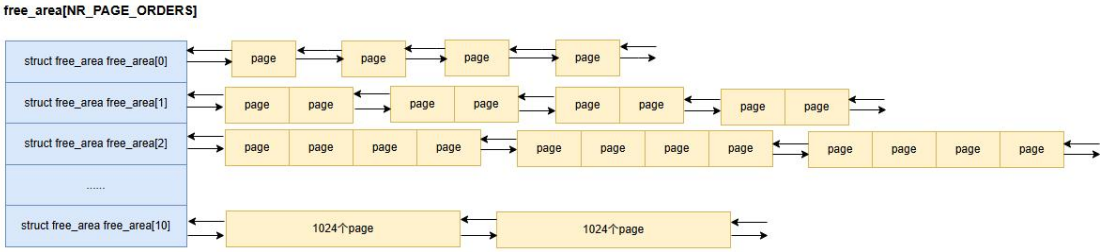
MIGRATE_ISOLATE 则是一个虚拟区域，用于跨越 NUMA 节点移动物理内存页，内核可以将物理内存页移动到使用该页最频繁的 CPU 所在的 NUMA 节点中。

设置页面迁移类型的原因，是为了解决前面我们反复提到的内存碎片问题，当系统长时间运行之后，随着不同尺寸内存的分配和释放，就会引起内存碎片，这些碎片会导致内核在明明还有足够内存的前提下，仍然无法找到一块足够大的连续内存分配。

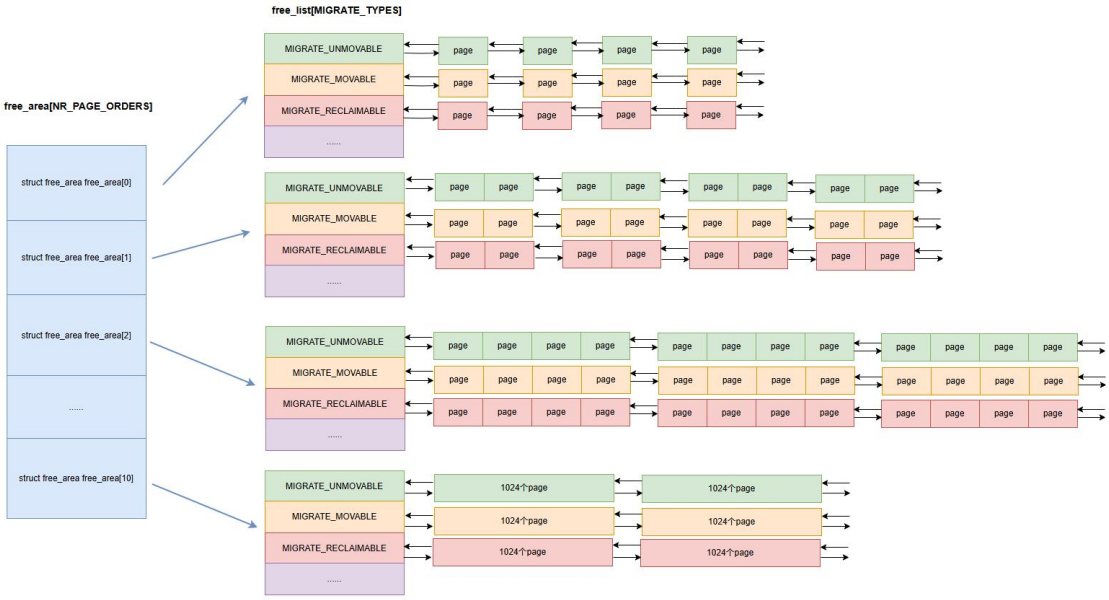
如果内存页是可以迁移的，内核就会将空闲的内存页迁移至一起，已分配的内存页迁移至一起，形成了一整块足够大的连续内存区域。如果这些内存页是可以回收的，内核也可以通过回收页面的方式，整理出一块足够大的空闲连续内存区域。



综上，首先伙伴系统会将物理内存区域 zone 中的空闲内存页按照分配阶 order 将相同尺寸的内存块组织在 free_area[MAX_ORDER] 数组中，如下图所示：



然后，在 struct free_area 结构中伙伴系统近一步根据这些相同尺寸内存块的页面迁移类型 MIGRATE_TYPES，将相同迁移类型的物理页面组织在 free_list[MIGRATE_TYPES] 数组中，最终形成了完整的伙伴系统结构，如下图所示：



6.2 伙伴系统的内存分配原理

内核中的伙伴指的是大小相同并且在物理内存上是连续的两个或者多个 page。比如在上图中，free_area[1] 中组织的是分配阶 order = 1 的内存块，内存块中包含了两个连续的空闲 page。这两个空闲 page 就是伙伴。free_area[10] 中组织的是分配阶 order = 10 的内存块，内存块中包含了 1024 个连续的空闲 page。这 1024 个空闲 page 就是伙伴。

再如下图中的 page0 和 page 1 是伙伴，page2 到 page 5 是伙伴，page6 到 page7 又是伙伴。但是 page0 和 page2 就不能成为伙伴，因为它们的物理内存是不连续的。同时 (page0 到 page3) 和 (page4 到 page7) 所组成的两个内存块又能构成一个伙伴。伙伴必须是大小相同并且在物理内存上是连续的两个或者多个 page。

page0	page1	page2	page3	page4	page5	page6	page7

内核可以通过如下接口向伙伴系统申请内存：

```
static inline struct page *alloc_pages(gfp_t gfp_mask, unsigned int order)
unsigned long __get_free_pages(gfp_t gfp_mask, unsigned int order)
unsigned long get_zeroed_page(gfp_t gfp_mask)
#define __get_dma_pages(gfp_mask, order) \
        __get_free_pages((gfp_mask) | GFP_DMA, (order))
```

1. 首先可以根据内存分配接口函数中的 gfp_t gfp_mask ，找到内存分配指定的 NUMA 节点和物理内存区域 zone ，然后找到物理内存区域 zone 对应的伙伴系统。

2. 随后内核通过接口中指定的分配阶 order，可以定位到伙伴系统的 free_area[order] 数组，其中存放的就是分配阶为 order 的全部内存块。

3. 最后内核进一步通过 gfp_t gfp_mask 掩码中指定的页面迁移类型 MIGRATE_TYPE，定位到 free_list[MIGRATE_TYPE]，这里存放的就是符合内存分配要求的所有内存块。通过遍历这个双向链表就可以轻松获得要分配的内存。

比如我们向内核申请 (2 ^ (order - 1)，2 ^ order] 之间大小的内存，并且这块内存我们指定的迁移类型为 MIGRATE_MOVABLE 时，内核会按照 2 ^ order 个内存页进行申请。

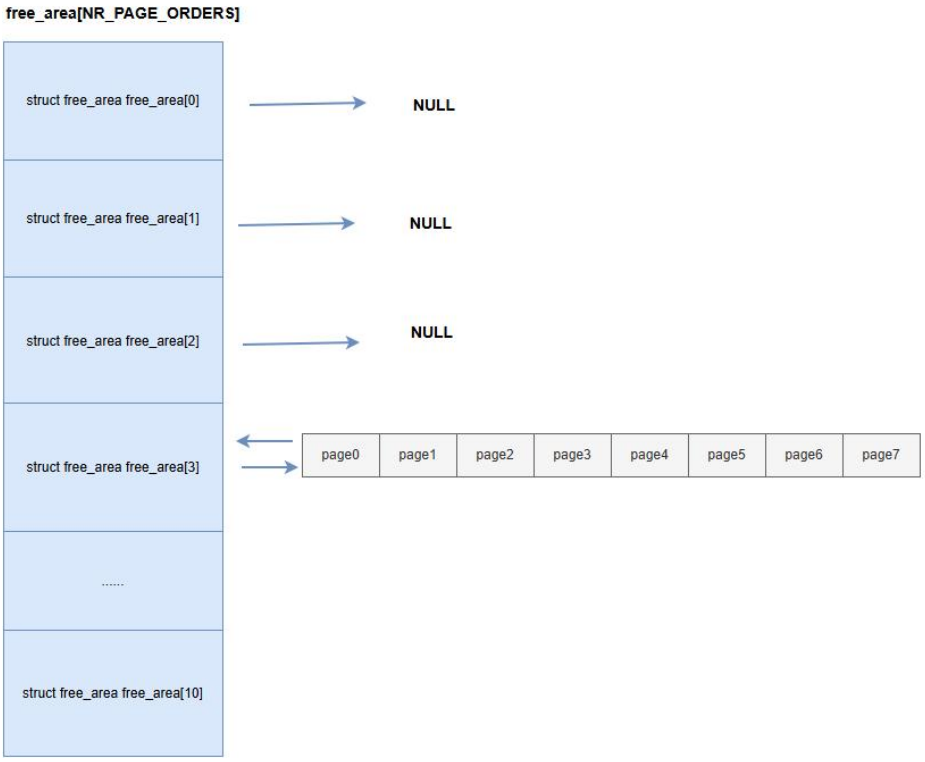
随后内核会根据 order 找到伙伴系统中的 free_area[order] 对应的 free_area 结构，并进一步根据页面迁移类型定位到对应的 free_list[MIGRATE_MOVABLE]，如果该迁移类型的 free_list 中没有空闲的内存块时，内核会进一步到上一级链表也就是 free_area[order + 1] 中寻找。

如果 free_area[order + 1] 中对应的 free_list[MIGRATE_MOVABLE] 链表中还是没有，则继续循环到更高一级 free_area[order + 2] 寻找，直到在 free_area[order + n] 中的 free_list[MIGRATE_MOVABLE] 链表中找到空闲的内存块。

但是此时在 free_area[order + n] 链表中找到的空闲内存块的尺寸是 2 ^ (order + n) 大小，而需要的是 2 ^ order 尺寸的内存块，于是内核会将这 2 ^ (order + n) 大小的内存块逐级减半分裂，将每一次分裂后的内存块插入到相应的 free_area 数组里对应的 free_list[MIGRATE_MOVABLE] 链表中，并将最后分裂出的 2 ^ order 尺寸的内存块分配给进程使用。

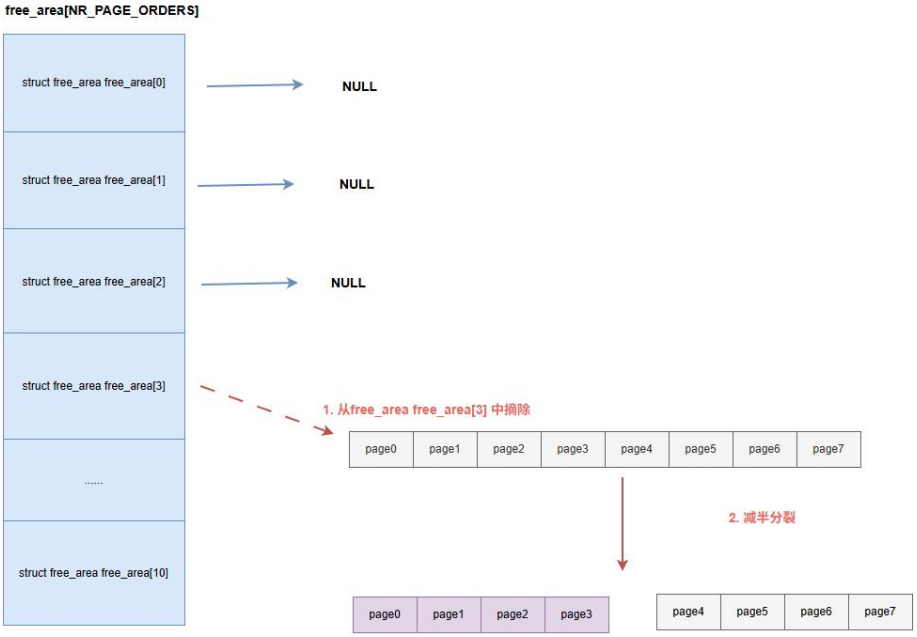
一个具体的例子来说明伙伴系统的整个内存分配过程：

我们假设当前伙伴系统中只有 order = 3 的空闲链表 free_area[3]，其余剩下的分配阶 order 对应的空闲链表中均是空的。 free_area[3] 中仅有一个空闲的内存块，其中包含了连续的 8 个 page。

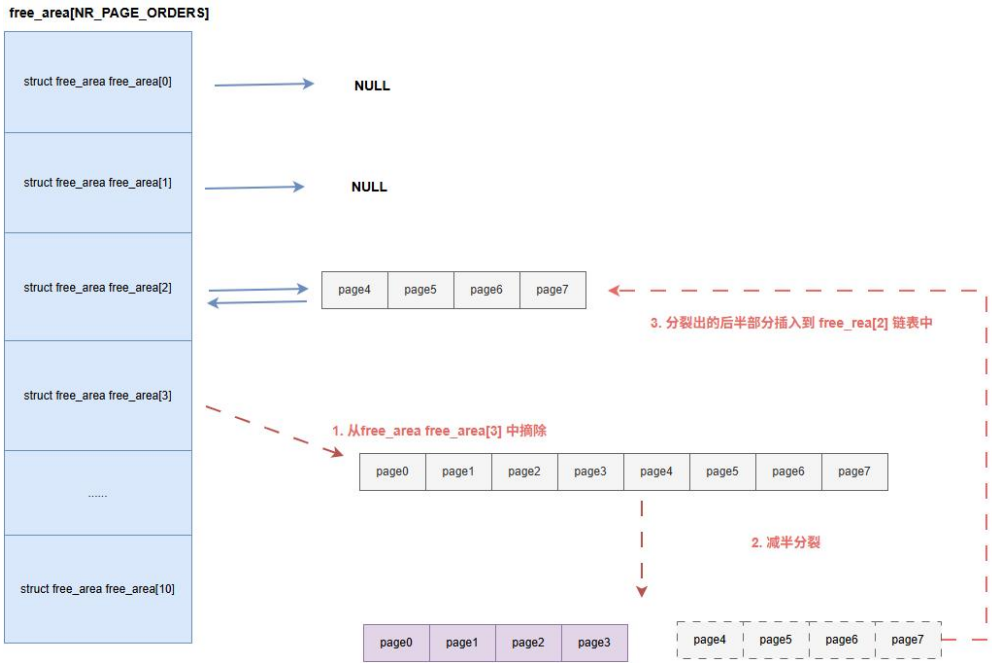


现在向伙伴系统申请一个 page 大小的内存（对应的分配阶 order = 0），那么内核会在伙伴系统中首先查看 order = 0 对应的空闲链表 free_area[0] 中是否有空闲内存块可供分配。

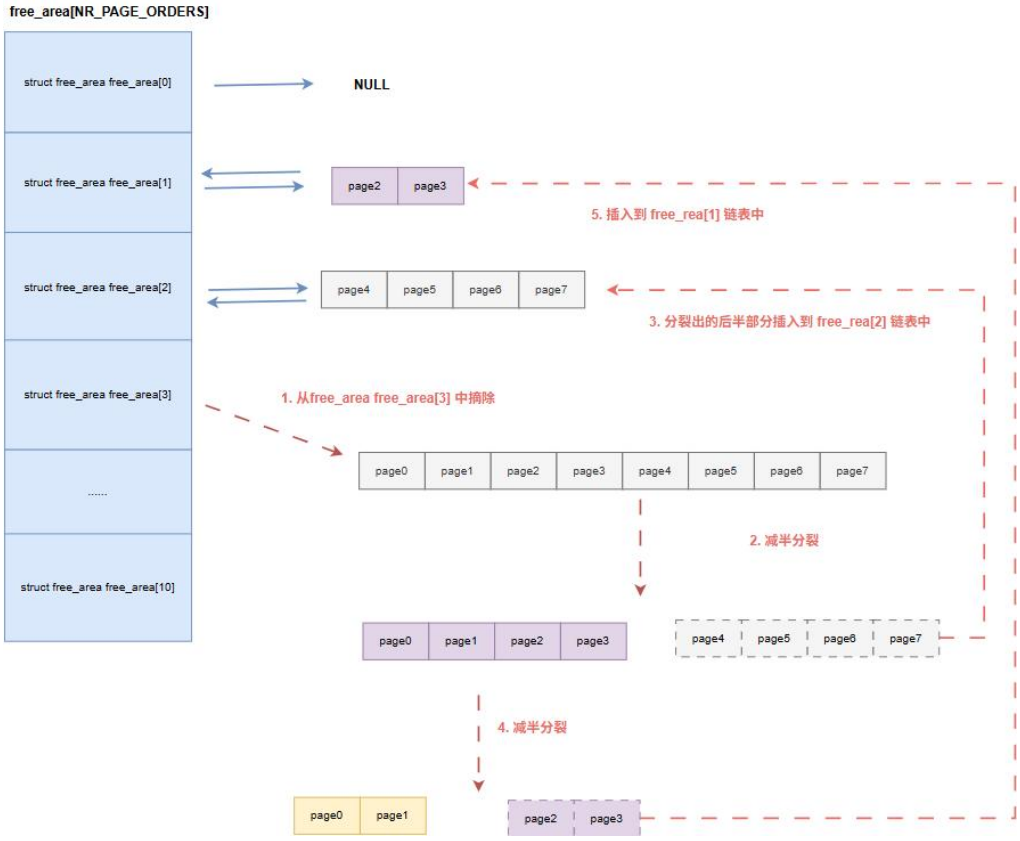
随后内核会根据前边介绍的内存分配逻辑，继续升级到 free_area[1]，free_area[2] 链表中寻找空闲内存块，直到查找到 free_area[3] 发现有一个可供分配的内存块。这个内存块中包含了 8 个 连续的空闲 page，但是此时申请的是一个 page。于是内核先将 free_area[3] 中的这个空闲内存块从链表中摘下，然后减半分裂成两个内存块，分裂出来的这两个内存块分别包含 4 个 page（分配阶 order = 2）。紫色色的代表原有内存块的前半部分，灰色代表原有内存块的后半部分。



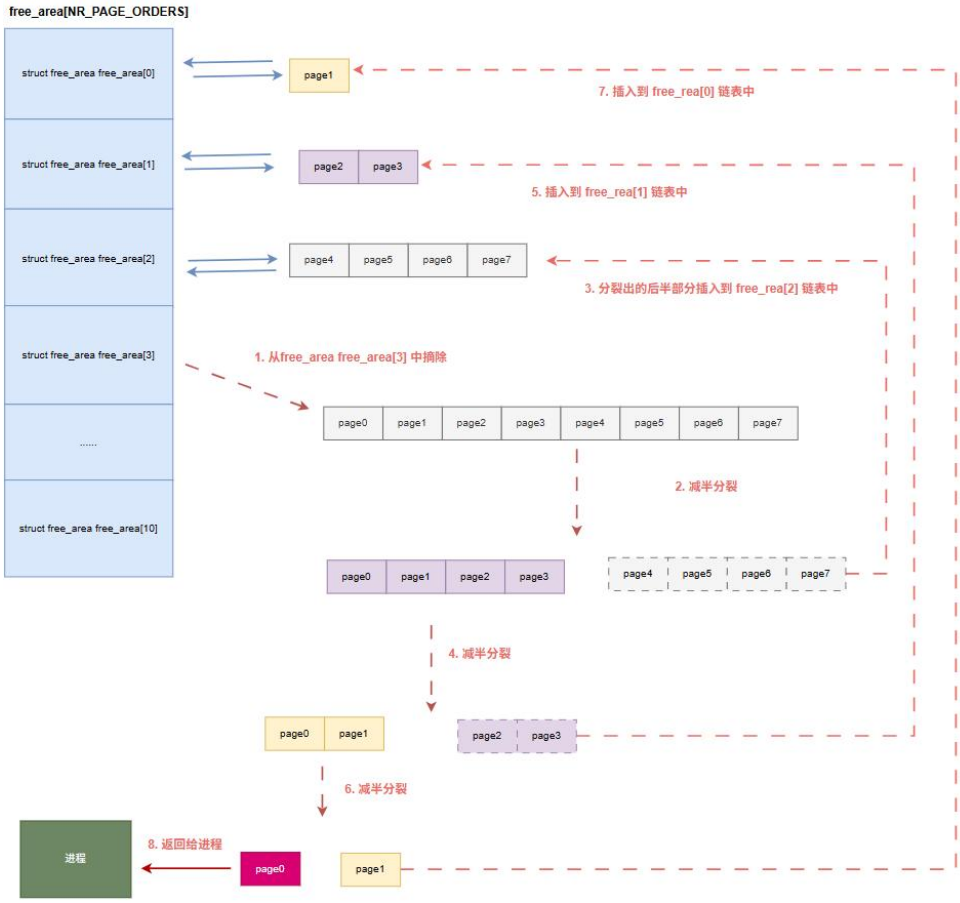
随后内核会将分裂出的后半部分（图中灰色部分，order = 2），插入到 free_rea[2] 链表中。



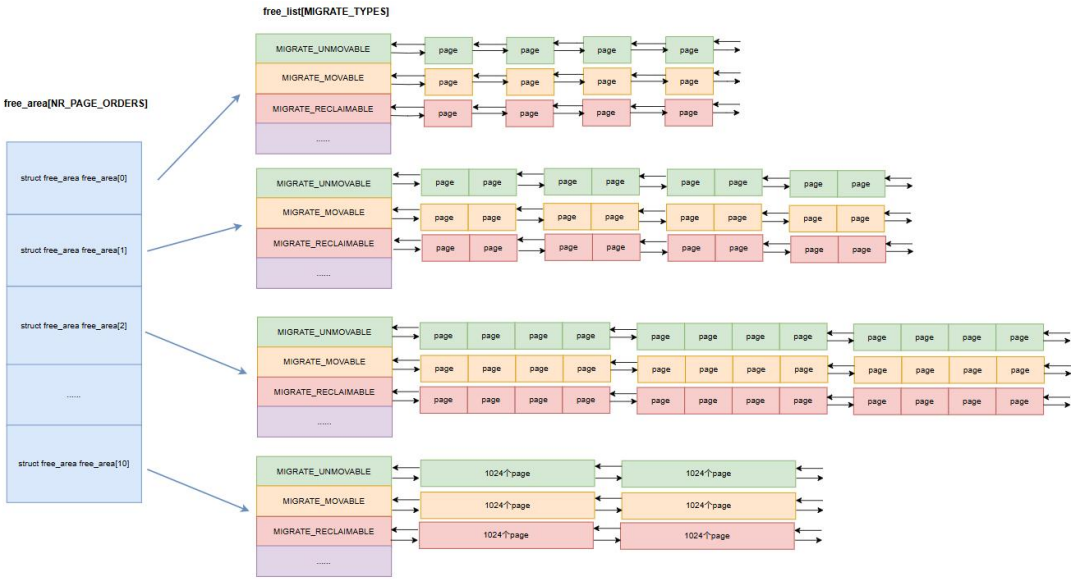
前半部分（图中紫色部分，order = 2）继续减半分裂，分裂出来的这两个内存块分别包含 2 个 page（分配阶 order = 1）。如下图中第 4 步所示，前半部分为黄色，后半部分为紫色。同理按照前边的分裂逻辑，内核会将后半部分内存块（紫色部分，分配阶 order = 1）插入到 free_area[1] 链表中。同理按照前边的分裂逻辑，内核会将后半部分内存块（紫色部分，分配阶 order = 1）插入到 free_area[1] 链表中，如下图，第 5 步所示。



前半部分（图中黄色部分，order = 1）继续减半分裂，分裂出来的这两个内存块分别包含 1 个 page（分配阶 order = 0），前半部分为红色，后半部分为黄色，如下图第 6 步所示。将后半部分插入到 frea_area[0] 链表中，如下如第 7 步所示。将前半部分返回给进程，如下图第 8 步所示。这时内存分配成功，流程结束。



以上流程就是伙伴系统的核心内存分配过程，把内存页面的迁移属性 MIGRATE_TYPES 考虑进来后，完整的伙伴系统内存分配流程如下：



加上内存 MIGRATE_TYPES 的组织结构后，分配流程还是和核心流程一样的，只是高阶 order 的减半分裂情形都发生在各个 free_area[order] 中固定的 free_list[MIGRATE_TYPE] 里罢了。比如分配的内存迁移属性要求是 MIGRATE_MOVABLE 类型，那么减半分裂流程分别发生在 free_area[2] ，free_area[1] ，free_area[0] 对应的 free_list[MIGRATE_MOVABLE] 中。

但是内存分配存在一种异常情形，比如想要分配特定迁移类型的内存，但是当前伙伴系统所有 free_area[order] 里对应的 free_list[MIGRATE_TYPE] 均无法满足内存分配的需求。此时，内核会跨越 NUMA 节点从其他节点上分配内存。

每个 NUMA 节点的 struct pglist_data 结构中都会包含一个 node_zonelists，其中包含了当前 NUMA 节点以及备用 NUMA 节点的所有内存区域以及对应的伙伴系统，当前 NUMA 节点内存不足时，内核会从 node_zonelists 中的备用 NUMA 节点中分配内存。

```
typedef struct pglist_data {
   ......省略......
    struct zone node_zones[MAX_NR_ZONES];
    struct zonelist node_zonelists[MAX_ZONELISTS];
    int nr_zones;
    ......省略......
}pg_data_t;
```

当伙伴系统中指定的迁移列表 free_list[MIGRATE_TYPE] 无法满足内存分配需求时，内核根据不同迁移类型也定义了不同的 fallback 规则：

```
/*
 * This array describes the order lists are fallen back to when
 * the free lists for the desirable migrate type are depleted
 *
 * The other migratetypes do not have fallbacks.
 */
static int fallbacks[MIGRATE_TYPES][MIGRATE_PCPTYPES - 1] = {
    [MIGRATE_UNMOVABLE]   = { MIGRATE_RECLAIMABLE, MIGRATE_MOVABLE   },
    [MIGRATE_MOVABLE]     = { MIGRATE_RECLAIMABLE, MIGRATE_UNMOVABLE },
    [MIGRATE_RECLAIMABLE] = { MIGRATE_UNMOVABLE,   MIGRATE_MOVABLE   },
};
```

比如：MIGRATE_UNMOVABLE 类型的 free_list 内存不足时，内核会 fallback 到 MIGRATE_RECLAIMABLE 中去获取，如果还是不足，则再次降级到 MIGRATE_MOVABLE 中获取，如果仍然无法满足内存分配，才会失败退出。

正常的分配流程先是从低阶到高阶依次查找空闲内存块，然后将高阶中的内存块依次减半分裂到低阶 free_list 链表中。内存分配 fallback 流程则刚好是相反的，它是先从备用 fallback 类型的迁移列表中的最高阶开始查找，找到一块空闲内存块之后，先迁移到最初指定的 free_list[MIGRATE_TYPE] 链表中，然后在指定的 free_list[MIGRATE_TYPE] 链表执行减半分裂。

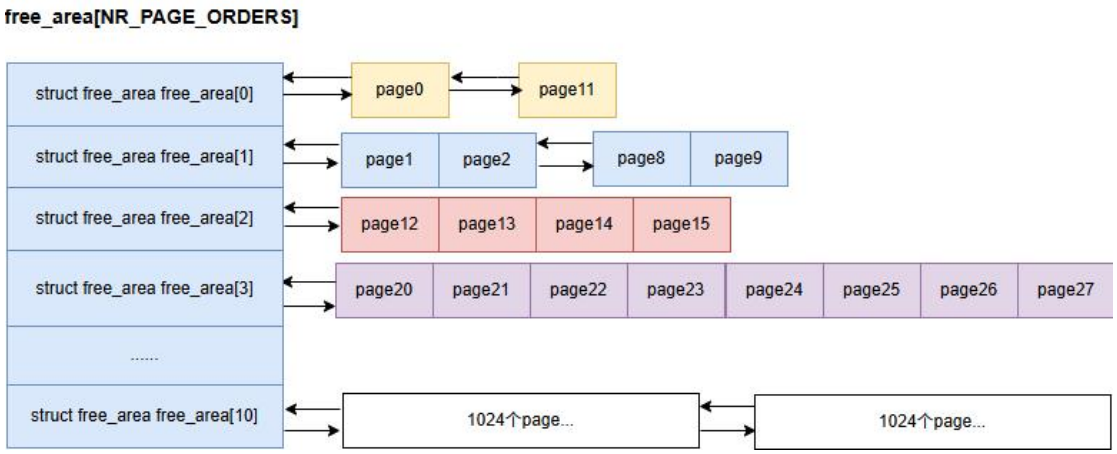
例如，向伙伴系统申请 MIGRATE_UNMOVABLE 迁移类型的内存时，假设内核在伙伴系统中的 free_area[0] 到 free_area[10] 中的所有 free_list[MIGRATE_UNMOVABLE] 链表中均无法找到一个空闲的内存块。那么就会 fallback 到 MIGRATE_RECLAIMABLE 类型，从最高阶 free_area[10] 中的 free_list[MIGRATE_RECLAIMABLE] 链表开始查找，如果找到一个空闲的内存块，则首先会迁移到对应的 order 的 free_list[MIGRATE_UNMOVABLE] 链表，然后流程继续回到核心流程，在各个 free_area[order] 对应的 free_list[MIGRATE_UNMOVABLE] 链表中

执行减半分裂。

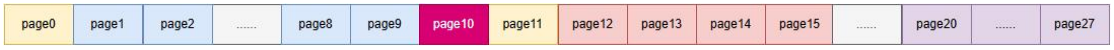
6.3 伙伴系统的内存回收原理

伙伴系统中的内存回收刚好和内存分配的过程相反，核心则是从低阶 free_list 中寻找释放内存块的伙伴，如果没有伙伴则将要释放的内存块插入到对应分配阶 order 的 free_list 中。如果存在伙伴，则将释放内存块与它的伙伴合并，作为一个新的内存块继续到更高阶的 free_list 中循环重复上述过程，直到不能合并为止。

一个伙伴系统内存回收的过程的例子，伙伴系统内存回收的过程：

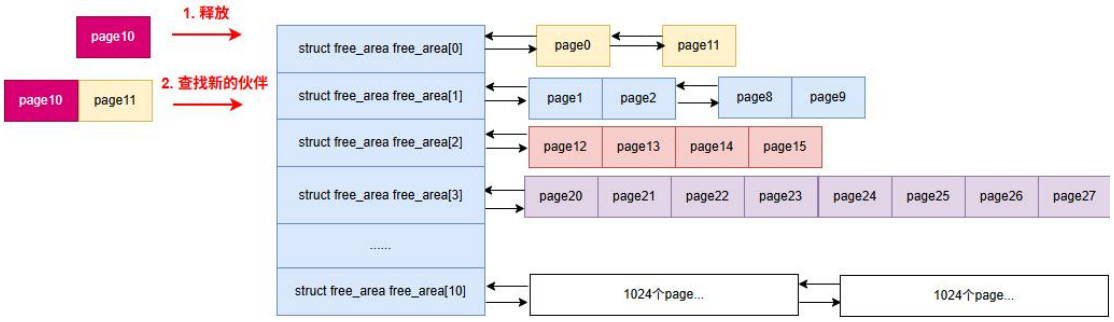


假设当前伙伴系统的状态如上图所示，向伙伴系统释放一个内存页（order = 0），编号为 10。伙伴系统中任何阶的单个空闲页块，其内部的物理页必然是连续的。上图内存页是被伙伴系统组织之后的视图，下面是物理内存页在物理内存上的真实视图（包含要被释放的内存页 10）：



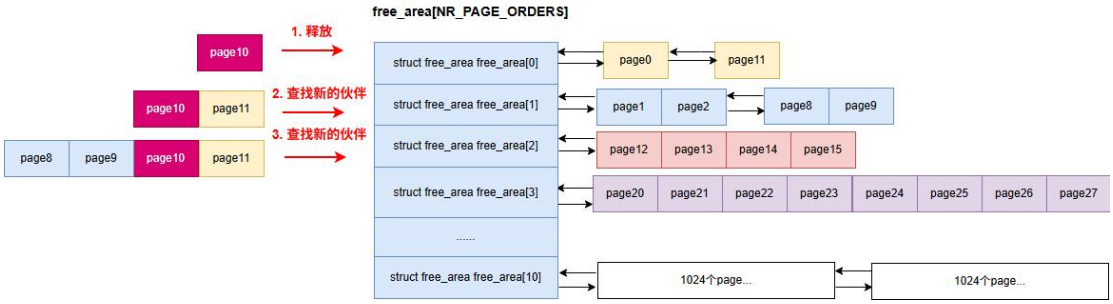
由于要释放的内存块只包含了一个物理内存页 page10，所以它的分配阶 order = 0，首先内核需要在伙伴系统 free_area[0] 中查找与 page10 大小相等并且连续的内存块（伙伴）。

从物理内存的真实视图中可以看到 page11 是 page10 的伙伴，于是将 page11 从 free_area[0] 上摘下并与 page10 合并组成一个新的内存块（分配阶 order = 1）。随后内核会在 free_area[1] 中查找新内存块的伙伴：

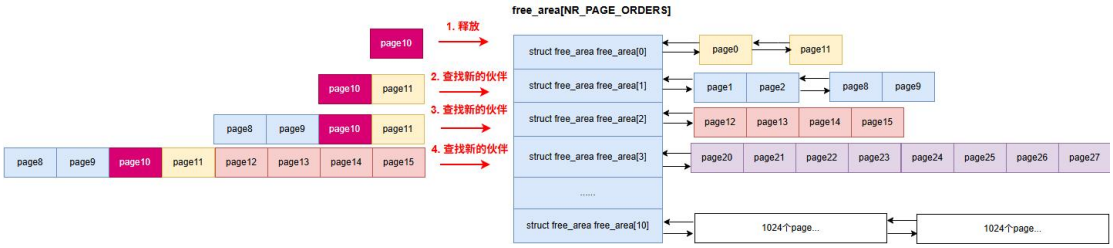


继续对比物理内存页的真实视图，发现在 free_area[1] 中 page8 和 page9 组成的内存

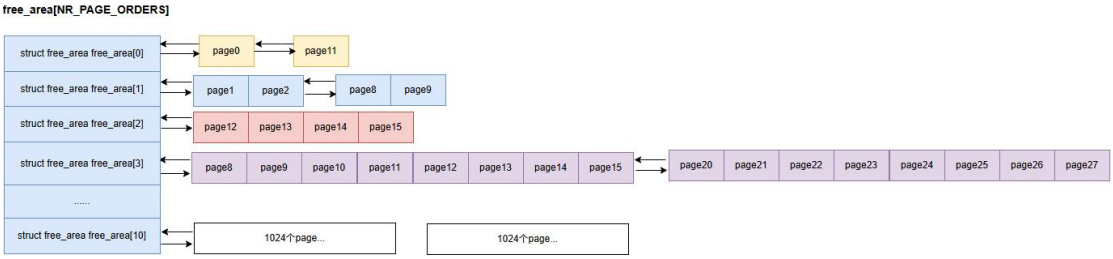
块与 page10 和 page11 组成的内存块是伙伴，于是继续将这两个内存块（分配阶 order = 1）继续合并成一个新的内存块（分配阶 order = 2）。随后内核会在 free_area[2] 中查找新内存块的伙伴：



继续对比物理内存页的真实视图，发现在 free_area[2] 中 page12，page13，page14，page15 组成的内存块与 page8，page9，page10，page11 组成的新内存块是伙伴，于是将它们从 free_area[2] 上摘下继续合并成一个新的内存块（分配阶 order = 3），随后内核会在 free_area[3] 中查找新内存块的伙伴：



对比物理内存页的真实视图，我们发现在 free_area[3] 中的内存块（page20 到 page 27）与新合并的内存块（page8 到 page15）虽然大小相同但是物理上并不连续，所以它们不是伙伴，不能在继续向上合并了。于是内核将 page8 到 pag15 组成的内存块（分配阶 order = 3）插入到 free_area[3] 中，至此内存释放过程结束，如下图。



内存分配是从高阶先查找到空闲内存块，然后依次减半分裂，将分裂后的内存块插入到低阶的 free_list 中，将最后分裂出来的内存块分配给进程。内存释放是先从低阶开始查找释放内存块的伙伴，如果找到，则两两合并成一个新的内存块，随后继续到高阶中去查找新内存块的伙伴，直到没有伙伴可以合并。

6.4 伙伴系统的内核源码分析

6.4.1 get_page_from_freelist 函数分析

伙伴系统的具体内存分配逻辑包含在 get_page_from_freelist 函数中，核心逻辑是：遍

历 struct alloc_context 里的 zonelist，挨个检查各个 NUMA 节点中的物理内存区域是否有足够的空闲内存可以满足本次的内存分配要求，如果可以满足则进入该物理内存区域的伙伴系统中完整真正的内存分配动作。

无论是快速路径还是慢速路径下的内存分配都需要最终调用 get_page_from_freelist 函数进行最终的内存分配。只不过，不同路径下 get_page_from_freelist 函数的内存分配策略以及需要考虑的内存水位线会有所不同，其中慢速路径下的内存分配策略会更加激进一些。

在每次调用 get_page_from_freelist 函数之前，内核都会根据新的内存分配策略来重新初始化 struct alloc_context 结构，alloc_context 结构体中包含了内存分配所需要的所有核心参数。

```
struct alloc_context {
    //运行进程 CPU 所在 NUMA 节点以及
    //其所有备用 NUMA 节点中允许内存分配的内存区域
    struct zonelist *zonelist;
    // NUMA 节点状态掩码
    nodemask_t *nodemask;
    // 内存分配优先级最高的内存区域 zone
    struct zoneref *preferred_zoneref;
    // 物理内存页的迁移类型分为：不可迁移，可回收，可迁移类型，防止内存碎片
    int migratetype;
     // 内存分配最高优先级的内存区域 zone
    enum zone_type highest_zoneidx;
    // 是否允许当前 NUMA 节点中的脏页均衡扩散迁移至其他 NUMA 节点
    bool spread_dirty_pages;
};
```

其中，

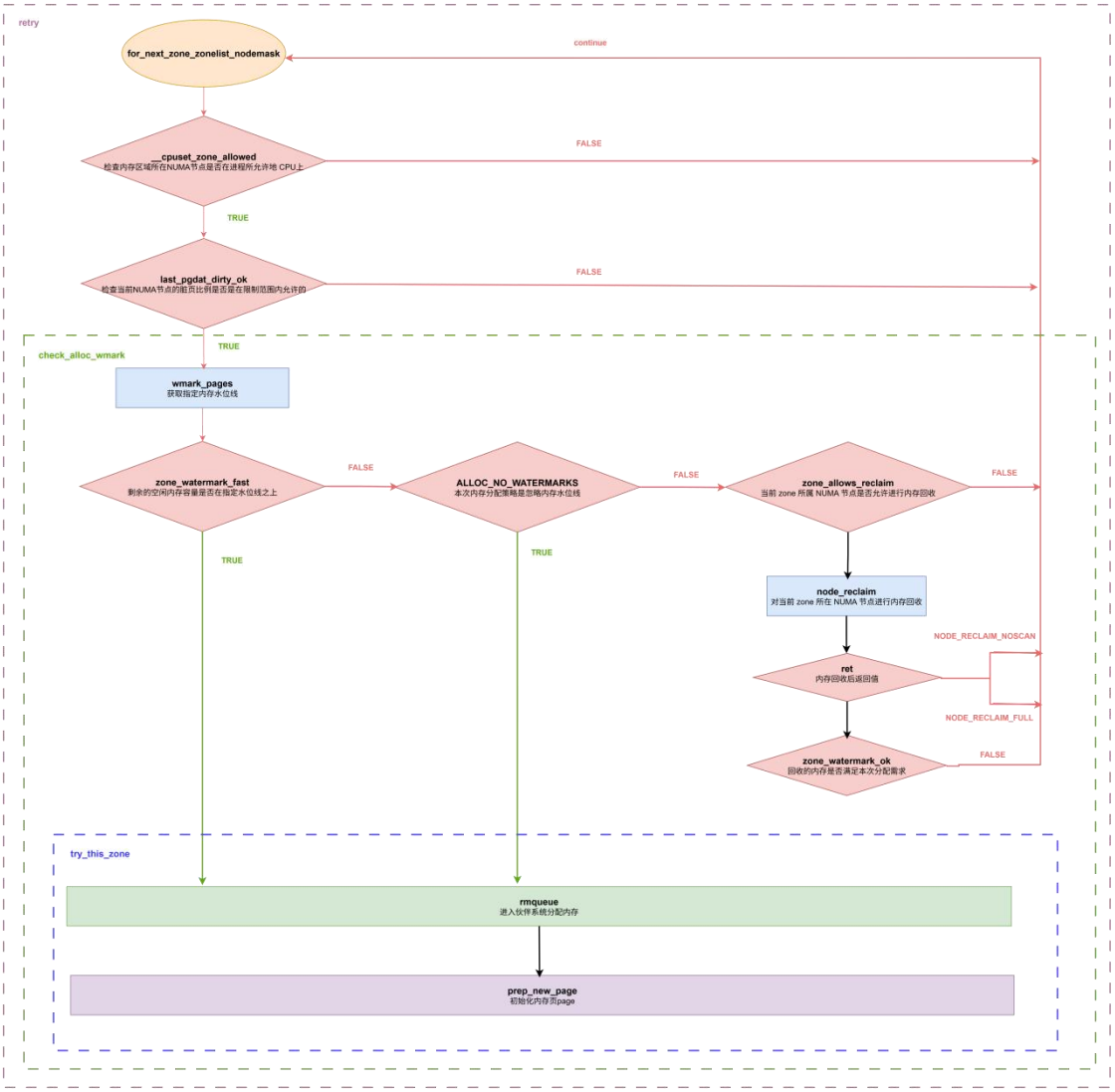
preferred_zoneref 表示当前本地 NUMA 节点（优先级最高）

zonelist 里面包含了当前 NUMA 节点在内的所有备用 NUMA 节点的所有物理内存区域，用于当前 NUMA 节点没有足够空闲内存的情况下进行跨 NUMA 节点分配。

struct pglist_data 里也有 numa 节点的备用列表 node_zonelists。node_zonelists 是一个全集，而 struct alloc_context 里的 zonelist 是在内存分配过程中，根据指定的内存分配策略从全集 node_zonelists 过滤出来的一个子集（允许进行本次内存分配的所有 NUMA 节点及其内存区域）。

```
typedef struct pglist_data {
    struct zone node_zones[MAX_NR_ZONES];  // NUMA 节点中的物理内存区域
    struct zonelist node_zonelists[MAX_ZONELISTS]; // NUMA 节点的备用列表
    int nr_zones; // NUMA 节点中的物理内存区域个数
} pg_data_t;
```

get_page_from_freelist 函数主要逻辑执行流程图：



get_page_from_freelist 函数具体实现：

```
/*
 * get_page_from_freelist goes through the zonelist trying to allocate
 * a page.
 */
static struct page *
get_page_from_freelist(gfp_t gfp_mask, unsigned int order, int alloc_flags,
                       const struct alloc_context *ac)
{
    struct zoneref *z;
     // 当前遍历到的内存区域 zone 引用
    struct zone *zone;

     // 最近遍历的 NUMA 节点
    struct pglist_data *last_pgdat = NULL;
```

```
    // 最近遍历的 NUMA 节点中包含的脏页数量是否在内核限制范围内
    bool last_pgdat_dirty_ok = false;

    // 如果需要避免内存碎片，则 no_fallback = true
    bool no_fallback;

retry:
    // 防止内存页块（pageblock）类型混合，减少永久性内存碎片
    no_fallback = alloc_flags & ALLOC_NOFRAGMENT;
    z = ac->preferred_zoneref;

    // 开始遍历 zonelist，查找可以满足本次内存分配的物理内存区域 zone
    for_next_zone_zonelist_nodemask(zone, z, ac->highest_zoneidx,
                    ac->nodemask) {

        // 指向分配成功之后的内存
        struct page *page;

        // 内存分配过程中设定的水位线
        unsigned long mark;

        // 检查内存区域所在 NUMA 节点是否在进程所允许的 CPU 上
        if (cpusets_enabled() &&
            (alloc_flags & ALLOC_CPUSET) &&
            !__cpuset_zone_allowed(zone, gfp_mask))
                continue;
        //每个 NUMA 节点中包含的脏页数量都有一定的限制。
        // 如果本次内存分配是为 page cache 分配的 page，
        //用于写入数据（不久就会变成脏页）
        // 这里需要检查当前 NUMA 节点的脏页比例是否在限制范围内允许的
        // 如果没有超过脏页限制则可以进行分配，
        //如果已经超过 last_pgdat_dirty_ok = false
        if (ac->spread_dirty_pages) {
            if (last_pgdat != zone->zone_pgdat) {
                last_pgdat = zone->zone_pgdat;
                last_pgdat_dirty_ok = node_dirty_ok(zone->zone_pgdat);
            }
            if (!last_pgdat_dirty_ok)
                continue;
        }
        // 如果内核设置了避免内存碎片标识，
        //在本地节点无法满足内存分配的情况下(因为需要避免内存碎片)
        // 这轮循环会遍历 remote 节点（跨 NUMA 节点）
```

```
        if (no_fallback && nr_online_nodes > 1 &&
            zone != ac->preferred_zoneref->zone) {
            int local_nid;
            // 如果本地节点分配内存失败是因为避免内存碎片的原因，
             //那么会继续回到本地节点进行 retry 重试
             //同时取消 ALLOC_NOFRAGMENT（允许引入碎片）
            local_nid = zone_to_nid(ac->preferred_zoneref->zone);
            if (zone_to_nid(zone) != local_nid) {
                // 内核认为保证本地的局部性会比避免内存碎片更加重要
                alloc_flags &= ~ALLOC_NOFRAGMENT;
                goto retry;
            }
        }
        //检测 zone->flags 中，第 ZONE_BELOW_HIGH(2) 位是否为 1；
        //当前内存域（zone）的空闲物理页数量 < high 水印阈值”（内存轻度紧张）
        //时跳转 check_alloc_wmark。
        //为 0 则表示空闲页 ≥ high 水印（内存健康）。
        if (test_bit(ZONE_BELOW_HIGH, &zone->flags))
            goto check_alloc_wmark;

        // 获取当前内存区域 zone 中 WMARK_HIGH 的水位线值，
        mark = high_wmark_pages(zone);

          // 检查当前遍历到的 zone 里剩余的空闲内存容量
          //是否在指定水位线 mark 之上
         // 剩余内存容量在水位线之下返回 false
        if (zone_watermark_fast(zone, order, mark,
                    ac->highest_zoneidx, alloc_flags,
                    gfp_mask))

            //此时空闲内存容量在 WMARK_HIGH 的水位线值之上，说明
            //就在本次遍历到的 zone 里尝试分配内存
            goto try_this_zone;
        else
           //返回 false，将标志位 flags 的第 ZONE_BELOW_HIGH 位置为 1。
           //表示当前内存域（zone）的空闲物理页数量 < high 水印阈值”
          //（内存轻度紧张）
            set_bit(ZONE_BELOW_HIGH, &zone->flags);
check_alloc_wmark:
          /*流程走到此处表示前内存域（zone）的空闲物理页数量 < high 水印阈值
          ，内存轻度紧张*/
          // 获取本次内存分配需要考虑到的内存水位线，
          //快速路径下是 WMARK_LOW，慢速路径下是 WMARK_MIN
        mark = wmark_pages(zone, alloc_flags & ALLOC_WMARK_MASK);
```

```
        //检查当前遍历到的 zone 里剩余的空闲内存容量
        //是否在指定水位线 mark 之上，剩余内存容量在水位线之下返回 false
        if (!zone_watermark_fast(zone, order, mark,
                        ac->highest_zoneidx, alloc_flags,
                        gfp_mask)) {
            int ret;
            if (has_unaccepted_memory()) {
                if (try_to_accept_memory(zone, order))
                    goto try_this_zone;
            }
#ifdef CONFIG_DEFERRED_STRUCT_PAGE_INIT
            if (deferred_pages_enabled()) {
                if (_deferred_grow_zone(zone, order))
                    goto try_this_zone;
            }
#endif
            /* Checked here to keep the fast path fast */
            BUILD_BUG_ON(ALLOC_NO_WATERMARKS < NR_WMARK);
             //如果本次内存分配策略是忽略内存水位线，
             //那么就在本次遍历到的 zone 里尝试分配内存
            if (alloc_flags & ALLOC_NO_WATERMARKS)
                goto try_this_zone;

           // 如果本次内存分配不能忽略内存水位线的限制，
          //那么就会判断当前 zone 所属 NUMA 节点是否允许进行内存回收
            if (!node_reclaim_enabled() ||
                !zone_allows_reclaim(ac->preferred_zoneref->zone, zone))

           //不允许进行内存回收则继续遍历下一个 NUMA 节点的内存区域
                continue;
            // 针对当前 zone 所在 NUMA 节点进行内存回收
            ret = node_reclaim(zone->zone_pgdat, gfp_mask, order);
            switch (ret) {
            case NODE_RECLAIM_NOSCAN:
                 // 返回该值表示当前 NUMA 节点没有必要进行回收。
                 //比如快速分配路径下就不处理页面回收的问题
                continue;
            case NODE_RECLAIM_FULL:
                // 返回该值表示通过扫描之后发现当前 NUMA 节点
               //并没有可以回收的内存页
                     continue;
            default:
              // 该分支表示当前 NUMA 节点已经进行了内存回收操作
             // zone_watermark_ok 判断内存回收
```

```
            //是否回收了足够的内存能否满足内存分配的需要
                if (zone_watermark_ok(zone, order, mark,
                    ac->highest_zoneidx, alloc_flags))
                    goto try_this_zone;
                continue;
            }
        }
try_this_zone:
      //这里就是伙伴系统的入口，rmqueue 函数中封装的就是伙伴系统的核心逻辑
        page = rmqueue(ac->preferred_zoneref->zone, zone, order,
                gfp_mask, alloc_flags, ac->migratetype);
        if (page) {

            // 分配内存成功，初始化内存页 page
            prep_new_page(page, order, gfp_mask, alloc_flags);
            if (unlikely(alloc_flags & ALLOC_HIGHATOMIC))
                reserve_highatomic_pageblock(page, zone);
            return page;
        } else {
                ....... 省略 .....
            }
        }
    }
     // 内存分配失败
    return NULL;
}
```

get_page_from_freelist 核心逻辑：通过 for_next_zone_zonelist_nodemask 来遍历当前 NUMA 节点以及备用节点的所有内存区域（zonelist），然后逐个通过 zone_watermark_fast 检查这些内存区域 zone 中的剩余空闲内存容量是否在指定的水位线 mark 之上。如果满足水位线的要求则直接调用 rmqueue 进入伙伴系统分配内存，分配成功之后通过 prep_new_page 初始化分配好的内存页 page。

如果当前正在遍历的 zone 中剩余空闲内存容量在指定的水位线 mark 之下，就需要通过 node_reclaim 触发内存回收，随后通过 zone_watermark_ok 检查经过内存回收之后，内核是否回收到了足够的内存以满足本次内存分配的需要。如果内存回收到了足够的内存则 zone_watermark_ok = true 随后跳转到 try_this_zone 分支在本内存区域 zone 中分配内存。否则继续遍历下一个 zone。

6.4.2 获取内存区域 zone 里指定的内存水位线

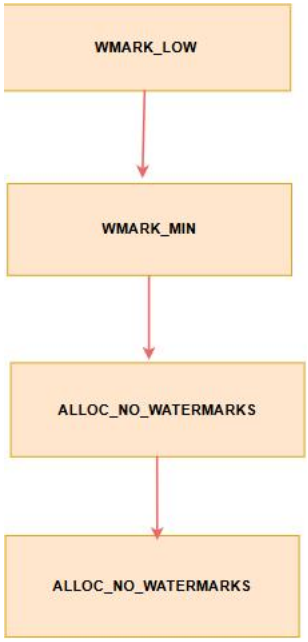
get_page_from_freelist 函数中的内存分配逻辑是要考虑内存水位线的，满足内存分配要求的物理内存区域 zone 中的剩余空闲内存容量必须在指定内存水位线之上。否则内核则认为内存不足不能进行内存分配。

在快速路径下，内存分配策略中的水位线设置为 WMARK_LOW；在慢速路径下，内存

分配策略中的水位线又被调整为了 WMARK_MIN。如果内存分配仍然失败，则内核会将内存分配策略中的水位线调整为 ALLOC_NO_WATERMARKS，表示再内存分配时可以忽略水位线的限制，再一次进行重试。

如果内存分配仍然失败，则内核会将内存分配策略中的水位线调整为 ALLOC_NO_WATERMARKS，表示再内存分配时，可以忽略水位线的限制，再一次进行重试。

根据分配策略，水位线调整过程：



不同的内存水位线会影响到内存分配逻辑，所以在通过 for_next_zone_zonelist_nodemask 遍历 NUMA 节点中的物理内存区域的一开始就需要获取该内存区域指定水位线的具体数值，内核通过 wmark_pages 宏来获取：

```
#define high_wmark_pages(z) (z->_watermark[WMARK_HIGH] +
z->watermark_boost)

struct zone {
     // 物理内存区域中的水位线
    unsigned long _watermark[NR_WMARK];

    // 优化内存碎片对内存分配的影响，可以动态改变内存区域的基准水位线。
    unsigned long watermark_boost;
};
```

6.4.3 检查 zone 中剩余内存容量是否满足水位线要求

通过 wmark_pages 获取到当前内存区域 zone 的指定水位线 mark 之后，需要近一步判断当前 zone 中剩余的空闲内存容量是否在水位线 mark 之上，这是保证内存分配顺利进行的必要条件。

内核中判断水位线的逻辑封装在 zone_watermark_fast 和 __zone_watermark_ok 函数

中，其中核心逻辑在 __zone_watermark_ok 里，zone_watermark_fast 只是用来快速检测分配阶 order = 0 情况下的相关水位线情况。

zone_watermark_fast 函数实现如下：

```
static inline bool zone_watermark_fast(struct zone *z, unsigned int order,
                unsigned long mark, int highest_zoneidx,
                unsigned int alloc_flags, gfp_t gfp_mask)
{
    long free_pages;
     // 获取当前内存区域中所有空闲的物理内存页
    free_pages = zone_page_state(z, NR_FREE_PAGES);
    // 快速检查分配阶 order = 0 情况下相关水位线，
    //空闲内存需要刨除掉为 highatomic 预留的紧急内存
    if (!order) {
        long usable_free;
        long reserved;
        // 可供本次内存分配使用的符合要求的真实可用内存，初始为 free_pages
        usable_free = free_pages;
         // 获取本次不能使用的空闲内存页数量
        reserved = __zone_watermark_unusable_free(z, 0, alloc_flags);
        // 计算真正可供内存分配的空闲页数量：空闲内存页全集 - 不能使用的空闲页
        usable_free -= min(usable_free, reserved);
        // 如果可用的空闲内存页数量大于内存水位线与预留内存之和
        // 那么表示物理内存区域中的可用空闲内存能够满足本次内存分配的需要
        if (usable_free > mark + z->lowmem_reserve[highest_zoneidx])
            return true;
    }
     // 近一步检查内存区域伙伴系统中是否有足够的 order 阶的内存块可供分配
    if (__zone_watermark_ok(z, order, mark, highest_zoneidx, alloc_flags,
                    free_pages))
        return true;
    if (unlikely(!order && (alloc_flags & ALLOC_MIN_RESERVE) &&
z->watermark_boost
        && ((alloc_flags & ALLOC_WMARK_MASK) == WMARK_MIN))) {
        mark = z->_watermark[WMARK_MIN];
        return __zone_watermark_ok(z, order, mark, highest_zoneidx,
                    alloc_flags, free_pages);
    }
    return false;
}
```

首先通过 zone_page_state 访问 vm_stat 来获取当前 zone 中剩余空闲内存页的总体容量 free_pages。需要注意，free_pages 表示的当前 zone 里剩余空闲内存页的一个总量，是一个全集的概念。其中还包括了内存区域的预留内存 lowmem_reserve 以及为 highatomic 预留的紧急内存。这些预留内存都有自己特定的用途，普通内存的申请不会用到

预留内存。

流程如果进入到 if (!order) 分支的话表示本次内存分配只是申请一个（order = 0）空闲的内存页，在这里会快速的检测相关水位线情况是否满足，如果满足就会快速返回。此时会用到两个局部变量：

usable_free：表示可供本次内存分配使用的空闲内存页总量。前边提到 free_pages 表示的是剩余空闲内存页的一个全集，里边还包括很多不能进行普通内存分配的空闲内存页，比如预留内存和紧急内存。

reserved：表示本次内存分配不能使用到的空闲内存页数量，这一部分的内存页数量计算是通过 __zone_watermark_unusable_free 函数完成的。最后使用 free_pages 减去 reserved 就可以得到真正的 usable_free 。

```
static inline long __zone_watermark_unusable_free(struct zone *z,
                unsigned int order, unsigned int alloc_flags)
{
    long unusable_free = (1 << order) - 1;

    if (likely(!(alloc_flags & ALLOC_RESERVES)))
        // 不可用内存的数量需要统计上 high-atomic 这部分内存
        unusable_free += z->nr_reserved_highatomic;
#ifdef CONFIG_CMA
      // 如果没有设置 ALLOC_CMA 则表示本次内存分配不能从 CMA 区域获取
    if (!(alloc_flags & ALLOC_CMA))

         // 不可用内存的数量需要统计上 CMA 区域中的空闲内存页
        unusable_free += zone_page_state(z, NR_FREE_CMA_PAGES);
#endif
#ifdef CONFIG_UNACCEPTED_MEMORY
    unusable_free += zone_page_state(z, NR_UNACCEPTED);
#endif
   // 返回不可用内存的数量，表示本次内存分配不能使用的内存容量
    return unusable_free;
}
```

如果 usable_free > mark + z->lowmem_reserve[highest_zoneidx] 条件为 true 表示当前可用剩余内存页容量在水位线 mark 之上，可以进行内存分配，返回 true。注意，判断可用内存是否满足水位线的关系时需要加上 **lowmem_reserve** 部分。

如果本次内存分配申请的是高阶内存块（ order > 0），则会进入 __zone_watermark_ok 函数中，近一步判断伙伴系统中是否有足够的高阶内存块能够满足 order 阶的内存分配：

```
bool __zone_watermark_ok(struct zone *z, unsigned int order, unsigned long
mark,
            int highest_zoneidx, unsigned int alloc_flags,
            long free_pages)
{
    // 保证内存分配顺利进行的最低水位线
     long min = mark;
```

```
    int o;

    // 获取真正可用的剩余空闲内存页数量
    free_pages -= __zone_watermark_unusable_free(z, order, alloc_flags);
    //如果设置了 ALLOC_RESERVES，表示可以在 min watermark 线之下分配内存
    if (unlikely(alloc_flags & ALLOC_RESERVES)) {
      // 如果又设置了 ALLOC_MIN_RESERVE 则将水位线降低二分之一，
      //使内存分配更加努力激进一些
        if (alloc_flags & ALLOC_MIN_RESERVE) {
            min -= min / 2;
            //如果又设置了 ALLOC_NON_BLOCK 则水位线再降低四分之一
            //使内存分配再激进些
            if (alloc_flags & ALLOC_NON_BLOCK)
                min -= min / 4;
        }
    // 在要进行 OOM 的情况下内存分配又会比上述策略更加努力激进一些，
    //所以这里水位线会再降低二分之一
        if (alloc_flags & ALLOC_OOM)
            min -= min / 2;
    }
     // 检查当前可用剩余内存是否在指定水位线之上。
    // 内存的分配必须保证可用剩余内存容量在指定水位线之上，否则不能进行内存分配
    if (free_pages <= min + z->lowmem_reserve[highest_zoneidx])
        return false;
   // 流程走到这里，对应内存分配阶 order = 0 的情况下就已经 OK 了
   // 剩余空闲内存在水位线之上，那么肯定能够分配一页出来
    if (!order)
        return true;

     // 但是对于 high-order 的内存分配，这里还需要近一步检查伙伴系统
     //根据伙伴系统内存分配的原理，
//这里需要检查高阶 free_list 中是否有足够的空闲内存块可供分配
    for (o = order; o < NR_PAGE_ORDERS; o++) {

        // 从当前分配阶 order 对应的 free_area 中检查是否有足够的内存块
        struct free_area *area = &z->free_area[o];
        int mt;
        // 如果当前 free_area 中的 nr_free = 0
        //表示对应 free_list 中没有合适的空闲内存块
        // 那么继续到高阶 free_area 中查找
        if (!area->nr_free)
            continue;
        // 检查 free_area 中所有的迁移类型 free_list 是否有足够的内存块
        for (mt = 0; mt < MIGRATE_PCPTYPES; mt++) {
```

```
            if (!free_area_empty(area, mt))
                return true;
        }
#ifdef CONFIG_CMA
        // 如果内存分配指定需要从 CMA 区域中分配连续内存
        // 那么就需要检查 MIGRATE_CMA 对应的 free_list 是否是空
        if ((alloc_flags & ALLOC_CMA) &&
            !free_area_empty(area, MIGRATE_CMA)) {
            return true;
        }
#endif
        //如果设置了 ALLOC_HIGHATOMIC 和 ALLOC_OOM，则表示可以
        //从 HIGHATOMIC 区中的紧急预留内存中分配，检查对应 free_list
        if ((alloc_flags & (ALLOC_HIGHATOMIC|ALLOC_OOM)) &&
            !free_area_empty(area, MIGRATE_HIGHATOMIC)) {
            return true;
        }
    }
    // 伙伴系统中的剩余内存块无法满足 order 阶的内存分配
    return false;
}
```

ALLOC_RESERVES 宏定义如下：

```
#define ALLOC_RESERVES
(ALLOC_NON_BLOCK|ALLOC_MIN_RESERVE|ALLOC_HIGHATOMIC|ALLOC_OOM)
```

在 __zone_watermark_ok 函数的开始需要计算出真正可用的剩余内存 free_pages 。

```
    // 获取真正可用的剩余空闲内存页数量
    free_pages -= __zone_watermark_unusable_free(z, order, alloc_flags);
```

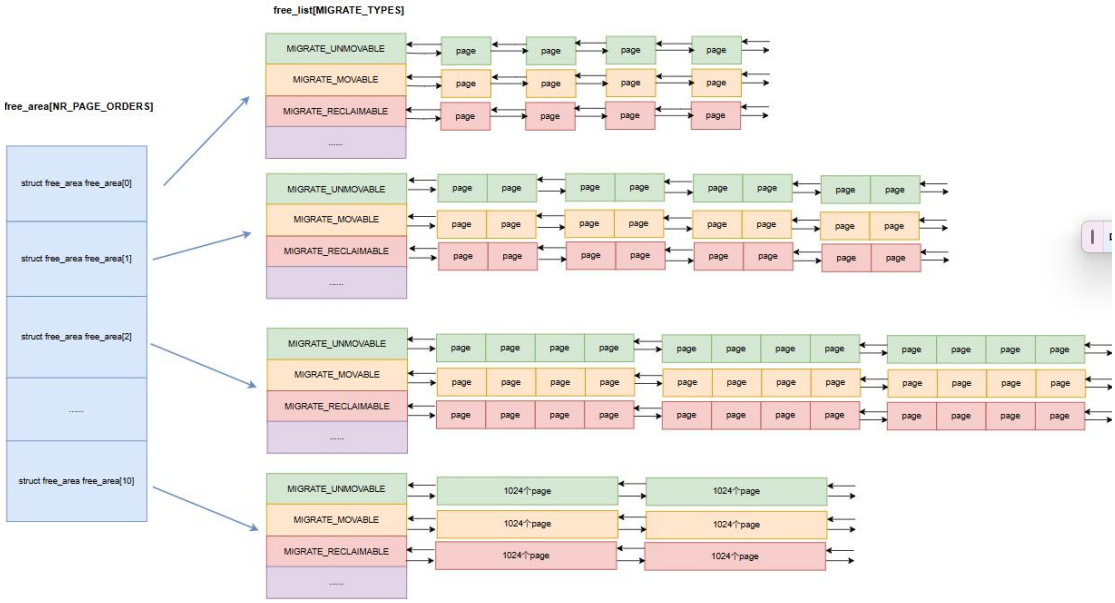
紧接着内核会根据 ALLOC_RESERVES、 ALLOC_MIN_RESERVE 、ALLOC_NON_BLOCK 和 ALLOC_OOM 标识来决定是否依次降低水位线的要求，使内存分配的策略越来越宽松，进而内存分配越来越激进。

再根据分配策略调整完水位线之后，需要判断当前可用剩余内存容量是否在水位线之上，如果是，则水位线检查完毕符合内存分配的要求。如果不是，则返回 false 不能进行内存分配。

```
free_pages <= min + z->lowmem_reserve[highest_zoneidx]
```

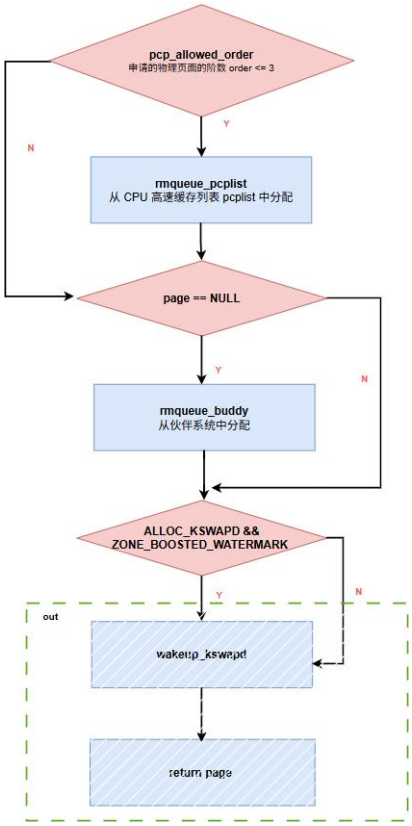
在水位线 OK 之后，对于 order = 0 的内存分配情形下，就已经 OK 了，可以放心直接进行内存分配了。但是对于 high-order 的内存分配情形，还需要近一步检查伙伴系统是否有足够的空闲内存块可以满足本次 high-order 的内存分配。

内核需要从当前分配阶 order 开始一直向高阶 free_area 中查找对应的 free_list 中是否有足够的内存块满足 order 阶的内存分配要求。如果有，那么水位线相关的校验工作到此结束，内核会直接去伙伴系统中申请 order 阶的内存块。如果没有，则水位线校验失败，伙伴系统无法满足本次的内存分配要求。



6.4.4 伙伴系统的实现

经过 zone_watermark_ok 的校验，现在内存水位线符合内存分配的要求，并且伙伴系统中有足够的空闲内存块可供内存分配申请，现在调用 rmqueue 函数进入伙伴系统进行内存分配。rmqueue 函数封装了伙伴系统的核心逻辑。rmqueue 函数处理逻辑流程图如下：



总体执行逻辑是：优先从低开销的 CPU 高速缓存列表中获取内存页，入口函数为：**rmqueue_pcplist**，失败后再走伙伴系统（buddy system）分配，入口函数为：**rmqueue_buddy**，

最后处理内存回收唤醒逻辑，具体实现如下：

```
/*
 * Do not instrument rmqueue() with KMSAN. This function may call
 * __msan_poison_alloca() through a call to set_pfnblock_flags_mask().
 * If __msan_poison_alloca() attempts to allocate pages for the stack depot,
it
 * may call rmqueue() again, which will result in a deadlock.
 */
__no_sanitize_memory
static inline
struct page *rmqueue(struct zone *preferred_zone,
           struct zone *zone, unsigned int order,
           gfp_t gfp_flags, unsigned int alloc_flags,
           int migratetype)
{
    struct page *page;
    WARN_ON_ONCE((gfp_flags & __GFP_NOFAIL) && (order > 1));
    if (likely(pcp_allowed_order(order))) {
       // 当我们申请的物理页面的阶数 order <= 3 时，
      //内核首先会从 CPU 高速缓存列表 pcplist 中直接分配，
      //而不会走伙伴系统，提高内存分配速度
        page = rmqueue_pcplist(preferred_zone, zone, order,
                       migratetype, alloc_flags);
        if (likely(page))
            goto out;
    }
    //当申请的阶数 order > 3 时,从伙伴系统中分配
    page = rmqueue_buddy(preferred_zone, zone, order, alloc_flags,
                         migratetype);
out:
    /* Separate test+clear to avoid unnecessary atomics */
    if ((alloc_flags & ALLOC_KSWAPD) &&
        unlikely(test_bit(ZONE_BOOSTED_WATERMARK, &zone->flags))) {
        clear_bit(ZONE_BOOSTED_WATERMARK, &zone->flags);
        wakeup_kswapd(zone, 0, 0, zone_idx(zone));
    }
    VM_BUG_ON_PAGE(page && bad_range(zone, page), page);
    return page;
}
```

6.4.5 从 CPU 高速缓存列表中获取内存页

内核对分配物理内存页的阶数小于等于 3 的情况做了特殊处理，这种情况下内核借助 CPU 高速缓存冷热页列表 pcplist 加速内存分配的处理，此时分配的内存页将来自于 pcplist 而不是伙伴系统。热页就是已经加载进 CPU 高速缓存中的物理内存页，冷页就是还未加载进 CPU 高速缓存中的物理内存页，冷页是热页的后备选项

pcp 是 per_cpu_pageset 的缩写，内核会为每个 CPU 分配一个高速缓存列表。在 NUMA 内存架构下，每个物理内存区域都归属于一个特定的 NUMA 节点，NUMA 节点中包含了一个或者多个 CPU，NUMA 节点中的每个内存区域会关联到一个特定的 CPU 上。而每个 CPU 都有自己独立的高速缓存，所以每个 CPU 对应一个 per_cpu_pageset 结构，用于管理这个 CPU 高速缓存中的冷热页。

每个 CPU 都可以访问系统中的所有物理内存页，尽管访问速度不同，因此特定的物理内存区域 struct zone 不仅要考虑到所属 NUMA 节点中相关的 CPU，还需要照顾到系统中的其他 CPU。

在 Linux 内核中，系统会经常请求和释放单个页面。如果针对每个 CPU，都为其预先分配一个高速缓存页列表，用于满足本地 CPU 发出的小数量页内存请求，就能提升系统的性能。所以在 struct zone 结构中持有了系统中所有 CPU 的高速缓存页列表 per_cpu_pageset。

```
struct zone {

        struct per_cpu_pages    __percpu *per_cpu_pageset;

}
struct per_cpu_pages {
    spinlock_t lock;    /* Protects lists field */
    int count;      /* pcplist 里的页面总数 */
    int high;       /* pcplist 里的高水位线，count 超过 high 时，
                       内核会释放 batch 个页面到伙伴系统中 */
    int high_min;       /* min high watermark */
    int high_max;       /* max high watermark */
    int batch;      /*pcplist 里的页面来自于伙伴系统，
                     batch 定义了每次从伙伴系统获取或者归还多少个页面*/
    u8 flags;       /* protected by pcp->lock */
    u8 alloc_factor;    /* batch scaling factor during allocate */
#ifdef CONFIG_NUMA
    u8 expire;      /* When 0, remote pagesets are drained */
#endif
    short free_count;   /* consecutive free count */

      // CPU 高速缓存列表 pcplist，每个迁移类型对应一个 pcplist*/
    struct list_head lists[NR_PCP_LISTS];
} ____cacheline_aligned_in_smp;
```

当内核尝试从 pcplist 中获取一个物理内存页时，会首先获取运行当前进程的 CPU 对

应的高速缓存列表 pcplist。然后根据指定的具体页面迁移类型 migratetype 获取对应迁移类型的 pcplist。当获取到符合条件的 pcplist 之后，内核会调用 __rmqueue_pcplist 从 pcplist 中摘下一个物理内存页返回。

```
/* Lock and remove page from the per-cpu list */
static struct page *rmqueue_pcplist(struct zone *preferred_zone,
            struct zone *zone, unsigned int order,
            int migratetype, unsigned int alloc_flags)
{
    struct per_cpu_pages *pcp;
    struct list_head *list;
    struct page *page;
    unsigned long __maybe_unused UP_flags;

    /* spin_trylock may fail due to a parallel drain or IRQ reentrancy. */
    pcp_trylock_prepare(UP_flags);

    // 获取运行当前进程的 CPU 高速缓存列表 pcplist
    pcp = pcp_spin_trylock(zone->per_cpu_pageset);
    if (!pcp) {
        pcp_trylock_finish(UP_flags);
        return NULL;
    }
    pcp->free_count >>= 1;
    // 获取指定页面迁移类型的 pcplist
    list = &pcp->lists[order_to_pindex(migratetype, order)];

    // 从指定迁移类型的 pcplist 中移除一个页面，用于内存分配
    page = __rmqueue_pcplist(zone, order, migratetype, alloc_flags, pcp,
list);
    pcp_spin_unlock(pcp);
    pcp_trylock_finish(UP_flags);
    if (page) {
        __count_zid_vm_events(PGALLOC, page_zonenum(page), 1 << order);
        // 统计内存区域内的相关信息
        zone_statistics(preferred_zone, zone, 1);
    }
    return page;
}
```

pcplist 中缓存的内存页面其实全部来自于伙伴系统，当 pcplist 中的页面数量 count 为 0 （表示此时 pcplist 里没有缓存的页面）时，内核会调用 rmqueue_bulk 从伙伴系统中获取 batch 个物理页面添加到 pcplist。

随后内核会将 pcplist 中的第一个物理内存页从链表中摘下返回，count 计数减一。

```
/* Remove page from the per-cpu list, caller must protect the list */
static inline
```

```
struct page *__rmqueue_pcplist(struct zone *zone, unsigned int order,
            int migratetype,
            unsigned int alloc_flags,
            struct per_cpu_pages *pcp,
            struct list_head *list)
{
    struct page *page;

    do {
    // 如果当前 pcplist 中的页面为空，
    //那么则从伙伴系统中获取 batch 个页面放入 pcplist 中
        if (list_empty(list)) {
            int batch = nr_pcp_alloc(pcp, zone, order);
            int alloced;
            alloced = rmqueue_bulk(zone, order,
                    batch, list,
                    migratetype, alloc_flags);
            pcp->count += alloced << order;
            if (unlikely(list_empty(list)))
                return NULL;
        }
         // 获取 pcplist 上的第一个物理页面
        page = list_first_entry(list, struct page, pcp_list);
         // 将该物理页面从 pcplist 中摘除
        list_del(&page->pcp_list);
         // pcplist 中的 count  减一
        pcp->count -= 1 << order;
    } while (check_new_pages(page, order));
    return page;
}
```

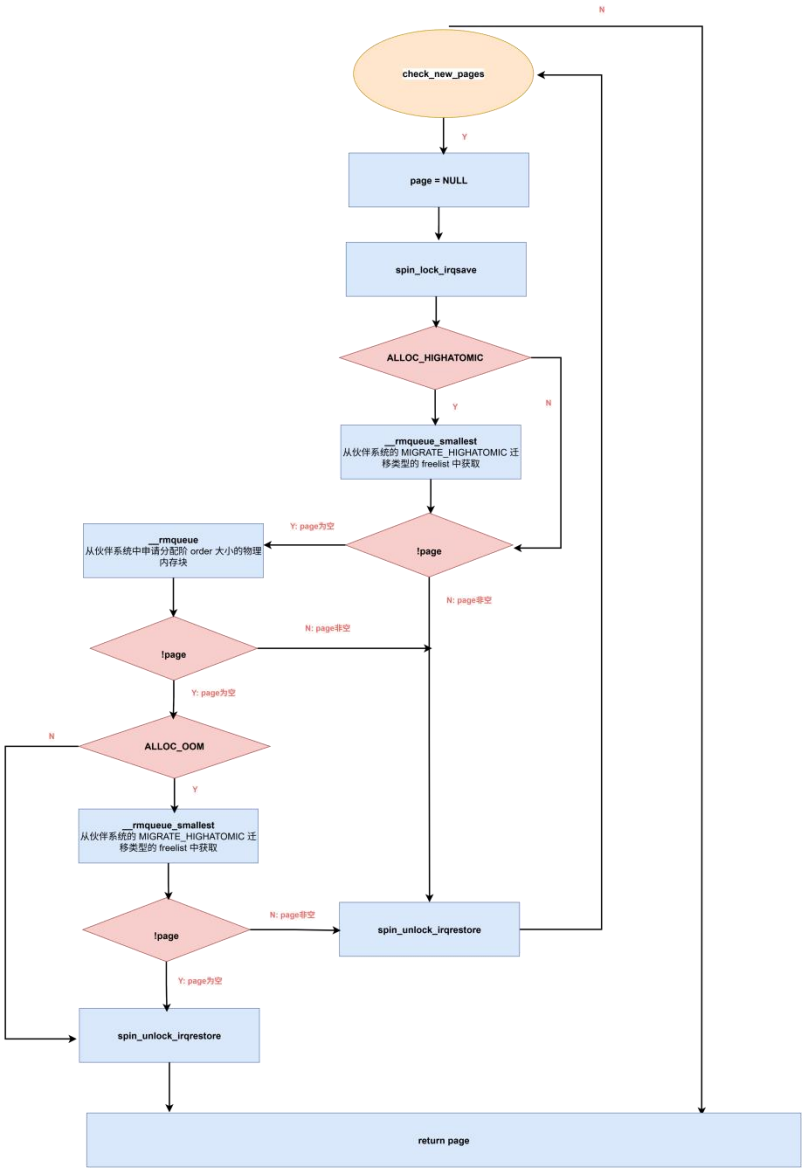
6.4.6 从伙伴系统中获取内存页

从伙伴系统中分配内存的入口函数是 rmqueue_buddy，具体实现如下：

```
static __always_inline
struct page *rmqueue_buddy(struct zone *preferred_zone, struct zone *zone,
              unsigned int order, unsigned int alloc_flags,
              int migratetype)
{
    struct page *page;
    unsigned long flags;

    do {
```

```
        page = NULL;
         // 加锁并关闭中断，防止并发访问
        spin_lock_irqsave(&zone->lock, flags);
        if (alloc_flags & ALLOC_HIGHATOMIC)
      // 如果设置了  ALLOC_HIGHATOMIC 分配策略，
      //则从伙伴系统的 HIGHATOMIC 迁移类型的 freelist 中获取
            page = __rmqueue_smallest(zone, order, MIGRATE_HIGHATOMIC);
        if (!page) {
          //从伙伴系统中申请分配阶 order 大小的物理内存块
            page = __rmqueue(zone, order, migratetype, alloc_flags);
            if (!page && (alloc_flags & ALLOC_OOM))
                page = __rmqueue_smallest(zone, order,
MIGRATE_HIGHATOMIC);
            if (!page) {
                // 解锁打开中断
                spin_unlock_irqrestore(&zone->lock, flags);
                 // 分配失败
                return NULL;
            }
        }
        __mod_zone_freepage_state(zone, -(1 << order),
                      get_pcppage_migratetype(page));
       // 解锁打开中断
        spin_unlock_irqrestore(&zone->lock, flags);
    } while (check_new_pages(page, order));
    __count_zid_vm_events(PGALLOC, page_zonenum(page), 1 << order);
     // 重新统计内存区域中的相关统计指标
    zone_statistics(preferred_zone, zone, 1);
    return page;
}
```

rmqueue_buddy 函数执行逻辑流程图如下：

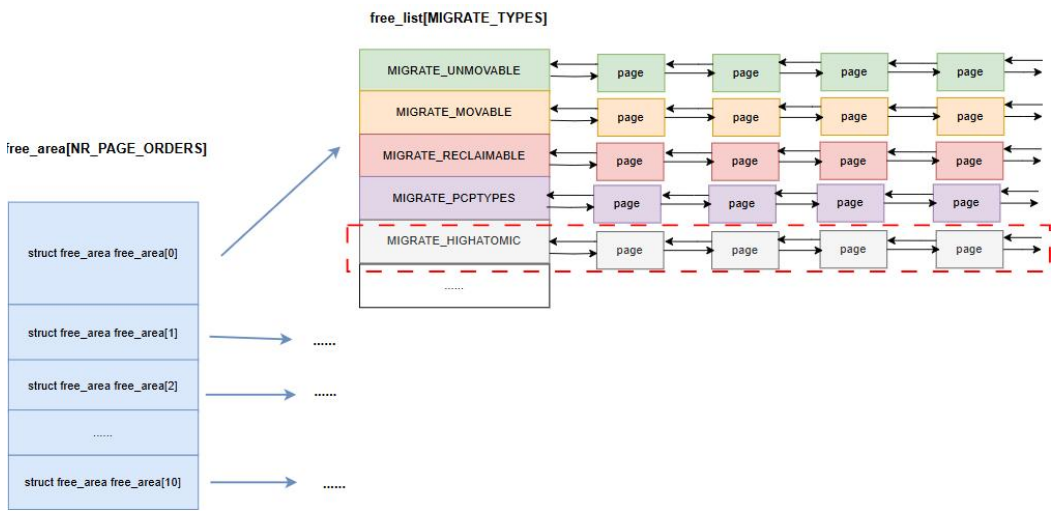


rmqueue_buddy 函数中，涉及到伙伴系统入口函数的有两个：

__rmqueue_smallest 函数主要是封装了整个伙伴系统关于内存分配的核心流程。

__rmqueue 函数封装的是伙伴系统的整个完整流程，底层调用了 __rmqueue_smallest 函数，它主要实现的是当伙伴系统 free_area 中对应的迁移列表 free_list[MIGRATE_TYPE] 无法满足内存分配需求时， 内存分配在伙伴系统中的 fallback 流程。

当我们向内核申请的内存页的阶数 order > 3 时，内核就会进入伙伴系统中为我们申请内存。如果内存分配策略 alloc_flags 指定了 ALLOC_HIGHATOMIC 时，就会调用 __rmqueue_smallest 直接进入伙伴系统，从 free_list[MIGRATE_HIGHATOMIC] 链表中分配 order 大小的物理内存块。



如果分配失败或者 alloc_flags 没有指定 ALLOC_HIGHATOMIC 则会通过 __rmqueue 进入伙伴系统，这里会处理分配失败之后的 fallback 逻辑。

```
static int fallbacks[MIGRATE_TYPES][MIGRATE_PCPTYPES - 1] = {
    [MIGRATE_UNMOVABLE]   = { MIGRATE_RECLAIMABLE, MIGRATE_MOVABLE   },
    [MIGRATE_MOVABLE]     = { MIGRATE_RECLAIMABLE, MIGRATE_UNMOVABLE },
    [MIGRATE_RECLAIMABLE] = { MIGRATE_UNMOVABLE,   MIGRATE_MOVABLE   },
};
```

6.4.6.1 __rmqueue_smallest 伙伴系统的核心实现

__rmqueue_smallest 函数的实现:

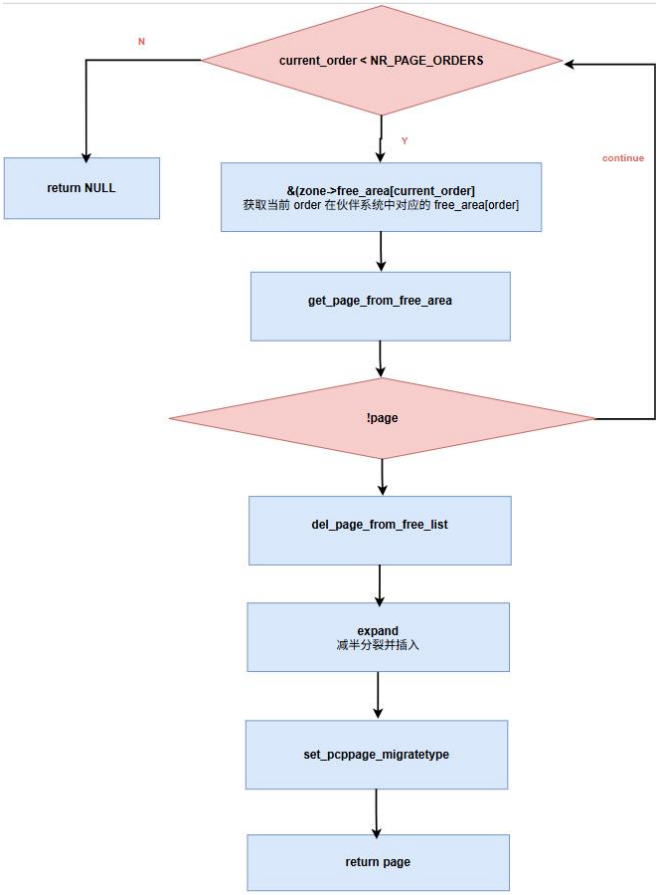
```
static __always_inline
struct page *__rmqueue_smallest(struct zone *zone, unsigned int order,
                        int migratetype)
{
    unsigned int current_order;
    struct free_area *area;
    struct page *page;
    /* 从当前分配阶 order 开始在伙伴系统对应的  free_area[order]  里查找合
    适尺寸的内存块*/
    for (current_order = order; current_order < NR_PAGE_ORDERS;
++current_order) {
        // 获取当前 order 在伙伴系统中对应的 free_area[order]
         area = &(zone->free_area[current_order]);

        // 从 free_area[order] 中对应的 free_list[MIGRATE_TYPE]
        //链表中获取空闲内存块
         page = get_page_from_free_area(area, migratetype);
         if (!page)
```

```
        // 如果当前 free_area[order] 中没有空闲内存块则继续向上查找
            continue;
        // 如果在当前 free_area[order] 中找到空闲内存块，
        //则从 free_list[MIGRATE_TYPE] 链表中摘除
        del_page_from_free_list(page, zone, current_order);
        // 将摘下来的内存块进行减半分裂并插入对应的尺寸的 free_area 中
        expand(zone, page, order, current_order, migratetype);
        // 设置页面的迁移类型
        set_pcppage_migratetype(page, migratetype);
        trace_mm_page_alloc_zone_locked(page, order, migratetype,
                pcp_allowed_order(order) &&
                migratetype < MIGRATE_PCPTYPES);
        // 内存分配成功返回
        return page;
    }
    // 内存分配失败返回 null
    return NULL;
}
```

减半分裂过程由 expand 函数实现。

```
static inline void expand(struct zone *zone, struct page *page,
    int low, int high, int migratetype)
{
    //当前要进行减半分裂的内存块的阶数，对应的物理内存页的数量
    unsigned long size = 1 << high;
    while (high > low) {
        //分配阶降级
        high--;
        //内存块尺寸减半(物理内存页的数减半)
        size >>= 1;
        VM_BUG_ON_PAGE(bad_range(zone, &page[size]), &page[size]);
        // 标记为保护页，当其伙伴被释放时，允许合并，
        if (set_page_guard(zone, &page[size], high, migratetype))
            continue;
        // 将本次减半分裂出来的第二个内存块插入到对应 free_area[high] 中
        add_to_free_list(&page[size], zone, high, migratetype);
         // 设置内存块的分配阶 high
        set_buddy_order(&page[size], high);
        // 本次分裂出来的第一个内存块继续循环进行减半分裂直到 high = low
        // 即已经分裂出来了指定 order 尺寸的内存块无需在进行分裂了，直接返回
    }
}
```

__rmqueue_smallest 函数执行逻辑流程图：



6.4.6.2 __rmqueue 伙伴系统的 fallback 实现

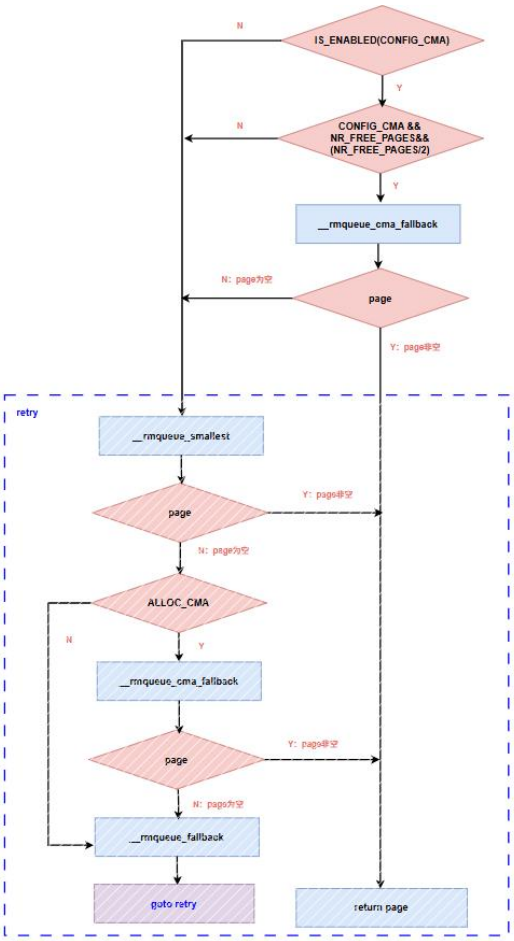
如果向内核申请的内存页的阶数 order > 3，并且内存分配策略 alloc_flags 中并没有设置 ALLOC_HIGHATOMIC 的时候，内存分配流程就会进入 __rmqueue 走常规的伙伴系统分配流程，主要包括“常规分配 → CMA fallback → 跨类型 fallback” 的三层逻辑：

第一层：精准匹配目标迁移类型。按指定 migratetype（迁移类型）从伙伴系统分配 2^order 连续页避免碎片；

第二层：动用 CMA 预留区域，兜底常规分配。平衡 CMA 与常规区域的可移动页分配；

第三层：跨类型分配。配失败时尝试跨迁移类型 fallback 重试，最后尝试避免分配失败；

__rmqueue 函数执行逻辑的流程如下：



__rmqueue 函数具体实现如下：

```
static __always_inline struct page *
__rmqueue(struct zone *zone, unsigned int order, int migratetype,
                        unsigned int alloc_flags)
{
    struct page *page;

    if (IS_ENABLED(CONFIG_CMA)) {
        if (alloc_flags & ALLOC_CMA &&
            zone_page_state(zone, NR_FREE_CMA_PAGES) >
            zone_page_state(zone, NR_FREE_PAGES) / 2) {
            page = __rmqueue_cma_fallback(zone, order);
            if (page)
                return page;
        }
    }
retry:
     // 首先进入伙伴系统到指定页面迁移类型的
    //free_list[migratetype] 获取空闲内存块
```

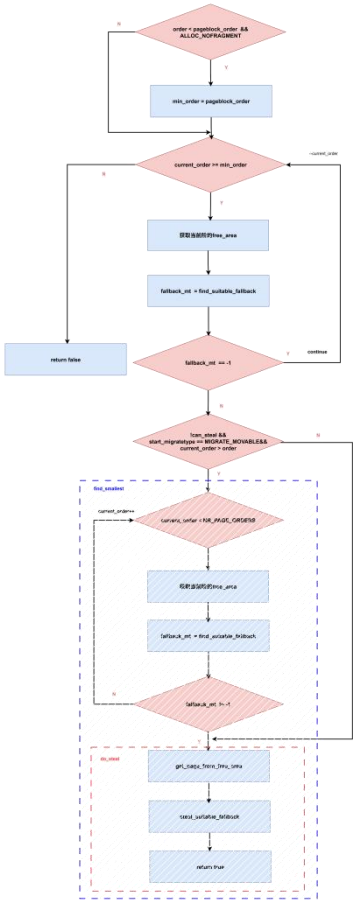
```
    page = __rmqueue_smallest(zone, order, migratetype);
    if (unlikely(!page)) {
    /*当伙伴系统中没有足够指定迁移类型 migratetype 的空闲内存块时，
    就会进入这个分支*/
      // 如果分配标志中含有  ALLOC_CMA
      //则优先 fallback 到 CMA 区中分配内存
        if (alloc_flags & ALLOC_CMA)
            page = __rmqueue_cma_fallback(zone, order);

      // 走常规的伙伴系统 fallback 流程，
        if (!page && __rmqueue_fallback(zone, order, migratetype,
                                        alloc_flags))
            goto retry;
    }
    // 内存分配成功
    return page;
}
```

该函数处理了伙伴系统内存分配的异常流程，即调用 __rmqueue_smallest 进入伙伴系统分配内存时，发现伙伴系统各个分配阶 free_area[order] 中对应的迁移列表 free_list[MIGRATE_TYPE] 无法满足内存分配需求时，__rmqueue_smallest 函数就会返回 null，伙伴系统内存分配失败。

随后内核就会进入伙伴系统的 fallback 流程，这里对 MIGRATE_MOVABLE 迁移类型做了一下特殊处理，当伙伴系统中 free_list[MIGRATE_MOVABLE] 没有足够空闲内存块时，会优先降级到 CMA 区域内进行分配。

```
static __always_inline struct page *__rmqueue_cma_fallback(struct zone
*zone,
                    unsigned int order)
{
    return __rmqueue_smallest(zone, order, MIGRATE_CMA);
}
```

如果指定的分配标志中不含& ALLOC_CMA 或者降级 CMA 之后仍然分配失败，内核就会进入 __rmqueue_fallback 走常规的 fallback 流程。__rmqueue_fallback 函数自行逻辑流程图如下：



在 __rmqueue_fallback 函数中，内核会根据预先定义的相关 fallback 规则开启内存分配的 fallback 流程。__rmqueue_fallback 函数具体实现：

```
static __always_inline bool
__rmqueue_fallback(struct zone *zone, int order, int start_migratetype,
                        unsigned int alloc_flags)
{
    // 最终 fall back 到伙伴系统的 free_area 中分配内存
    struct free_area *area;

    // fallback 和正常的分配流程正好相反，
    //是从最高阶的 free_area[MAX_ORDER - 1] 开始查找空闲内存块
    int current_order;

    //最初指定的内存分配阶
    int min_order = order;
    struct page *page;
    // 最终计算出 fallback 到这个页面迁移类型 free_list 上
    int fallback_mt;

    // 是否可以从 free_list[fallback]
    //中窃取内存块到 free_list[start_migratetype]  中
```

```
// start_migratetype 表示我们最初指定的页面迁移类型
bool can_steal;

// 如果设置了 ALLOC_NOFRAGMENT 表示不希望引入内存碎片
// 在这种情况下内核会更加倾向于分配一个尽可能大的内存块，
//避免向其他链表引入内存碎片
if (order < pageblock_order && alloc_flags & ALLOC_NOFRAGMENT)
// pageblock_order 用于定义系统支持的巨型页对应的分配阶
// 默认为最大分配阶 - 1 = 9
    min_order = pageblock_order;

 // fallback 内存分配流程从最高阶 free_area
  //开始查找空闲内存块（页面迁移类型为 fallback 类型）
for (current_order = MAX_PAGE_ORDER; current_order >= min_order;
            --current_order) {

    // 获取伙伴系统中 current_order 的 free_area
    //首次循环时，为最高阶的 free_area
    area = &(zone->free_area[current_order]);

    // 按照上述的内存分配 fallback 规则查找最合适的 fallback 迁移类型
    fallback_mt = find_suitable_fallback(area, current_order,
            start_migratetype, false, &can_steal);


     // 如果没有合适的 fallback_mt，
      //则继续降级到下一个分配阶 free_area 中查找
    if (fallback_mt == -1)
        continue;
      // can_steal 会在 find_suitable_fallback 的过程中被设置
     // 当我们指定的页面迁移类型为 MIGRATE_MOVABLE
      //并且无法从其他 fallback 迁移类型列表
      //中窃取页面 can_steal = false 时
     // 内核会更加倾向于 fallback 分配最小的可用页面，
    //即尺寸和指定 order 最接近的页面数量而不是尺寸最大的
     // 因为这里的条件是分配可移动的页面类型，
    //天然可以避免永久内存碎片，无需按照最大的尺寸分配
    if (!can_steal && start_migratetype == MIGRATE_MOVABLE
                && current_order > order)
        goto find_smallest;

    // can_steal = true，则开始从 free_list[fallback] 列表中窃取页面
    goto do_steal;
}
```

```
    return false;
find_smallest:
     // 该分支目的在于寻找尺寸最贴近指定 order 大小的最小可用页面
    // 从指定 order 开始 fallback
    for (current_order = order; current_order < NR_PAGE_ORDERS;
current_order++) {
        area = &(zone->free_area[current_order]);
        fallback_mt = find_suitable_fallback(area, current_order,
                start_migratetype, false, &can_steal);
        if (fallback_mt != -1)
            break;
    }
    VM_BUG_ON(current_order > MAX_PAGE_ORDER);
do_steal:
     // 从上述流程获取到的伙伴系统 free_area 中获取 free_list[fallback_mt]
    page = get_page_from_free_area(area, fallback_mt);
    // 从 free_list[fallback_mt]
    //中窃取页面到 free_list[start_migratetype] 中
    steal_suitable_fallback(zone, page, alloc_flags, start_migratetype,
                                can_steal);
    trace_mm_page_alloc_extfrag(page, order, current_order,
        start_migratetype, fallback_mt);

     // 返回到 __rmqueue 函数中进行 retry 重试流程,
    //此时 free_list[start_migratetype] 中已经有足够的内存页面可供分配了
    return true;
}
```

从上述内存分配 fallback 源码实现中，我们可以看出内存分配 fallback 流程正好和正常的分配流程相反：

伙伴系统正常内存分配流程先是从低阶到高阶依次查找空闲内存块，然后将高阶中的内存块依次减半分裂到低阶 free_list 链表中。

伙伴系统 fallback 内存分配流程则是先从最高阶开始查找，找到一块空闲内存块之后，先迁移到最初指定的 free_list[start_migratetype] 链表中，**然后在指定的 free_list[start_migratetype] 链表中执行减半分裂**。

fallback 核心的逻辑封装在 find_suitable_fallback 函数中:

1. fallback 规则定义在 fallbacks[MIGRATE_TYPES][MIGRATE_PCPTYPES - 1] 二维数组中，第一维表示要进行 fallback 的页面迁移类型 migratetype。第二维 migratetype 迁移类型可以 fallback 到哪些迁移类型中，这些可以 fallback 的页面迁移类型按照优先级排列。

```
static int fallbacks[MIGRATE_TYPES][MIGRATE_PCPTYPES - 1] = {
    [MIGRATE_UNMOVABLE]   = { MIGRATE_RECLAIMABLE, MIGRATE_MOVABLE   },
    [MIGRATE_MOVABLE]     = { MIGRATE_RECLAIMABLE, MIGRATE_UNMOVABLE },
    [MIGRATE_RECLAIMABLE] = { MIGRATE_UNMOVABLE,   MIGRATE_MOVABLE   },
};
```

2. 该函数的核心逻辑是在 for (i = 0;; i++) 循环中按照 fallbacks[migratetype][i] 数组定义的 fallback 优先级，依次在 free_area[order] 中对应的 free_list[fallback] 列表中查找是否有空闲的内存块。

3. 如果当前 free_list[fallback] 列表中没有空闲内存块，则继续在 for 循环中降级到下一个 fallback 页面迁移类型中去查找，也就是 for 循环中的 fallbacks[migratetype][i] 。直到找到空闲内存块为止，否则返回 -1。

```
int find_suitable_fallback(struct free_area *area, unsigned int order,
            int migratetype, bool only_stealable, bool *can_steal)
{
    int i;
    // 最终选取的 fallback 页面迁移类型
    int fallback_mt;

    // 当前 free_area[order] 中以无空闲页面，则返回失败
    if (area->nr_free == 0)
        return -1;
    *can_steal = false;
    // 按照 fallback 优先级，循环在 free_list[fallback]
    //中查询是否有空闲内存块
    for (i = 0; i < MIGRATE_PCPTYPES - 1 ; i++) {
        // 按照优先级获取 fallback 页面迁移类型
        fallback_mt = fallbacks[migratetype][i];
         // 如果当前 free_list[fallback]为空则继续循环降级查找
        if (free_area_empty(area, fallback_mt))
            continue;
         // 判断是否可以从 free_list[fallback]
         //窃取页面到指定 free_list[migratetype] 中
        if (can_steal_fallback(order, migratetype))
            *can_steal = true;
        if (!only_stealable)
            return fallback_mt;
        if (*can_steal)
            return fallback_mt;
    }
    return -1;
}
```

can_steal_fallback 函数中定义了是否可以从 free_list[fallback] 窃取页面到指定 free_list[migratetype] 中，逻辑如下：

1. 如果我们指定的内存分配阶 order 大于等于 pageblock_order，则返回 true。pageblock_order 表示系统中支持的巨型页对应的分配阶，默认为伙伴系统中的最大分配阶减一。

2. 如果我们指定的页面迁移类型为 MIGRATE_RECLAIMABLE 或者

MIGRATE_UNMOVABLE，则不管我们要申请的页面尺寸有多大，内核都会允许窃取页面 can_steal = true ，因为它们最终会 fallback 到 MIGRATE_MOVABLE 可移动页面类型中，这样造成内存碎片的情况会少一些。

3. 当内核全局变量 page_group_by_mobility_disabled 设置为 1 时，则所有物理内存页面都是不可移动的，这时内核也允许窃取页面。

在系统初始化期间，所有页都被标记为 MIGRATE_MOVABLE 可移动的页面类型，其他的页面迁移类型都是后来通过 __rmqueue_fallback 窃取产生的。

```
// 这里窃取页面的目的是从 fallback 类型的 freelist 中拿到一个高阶的大内存块
// 之所以要窃取尽可能大的内存块是为了避免引入内存碎片
// 但 MIGRATE_MOVABLE 类型的页面本身就可以避免永久内存碎片
// 所以 fallback MIGRATE_MOVABLE 类型的页面时，
//会跳转到 find_smallest 分支只需要选择一个合适的 fallback 内存块即可
static bool can_steal_fallback(unsigned int order, int start_mt)
{
    if (order >= pageblock_order)
        return true;

    if (order >= pageblock_order / 2 ||
        start_mt == MIGRATE_RECLAIMABLE ||
        start_mt == MIGRATE_UNMOVABLE ||
        page_group_by_mobility_disabled)
        return true;

   // 跳转到 find_smallest 分支选择一个合适的 fallback 内存块
    return false;
}
```

6.4.7 内存分配成功之后初始化 page

当通过 rmqueue 函数从伙伴系统中成功申请到分配阶为 order 大小的内存块时，内核需要调用 prep_new_page 函数初始化这部分内存块，之后才能返回给进程使用。

```
static void prep_new_page(struct page *page, unsigned int order, gfp_t
gfp_flags,
                          unsigned int alloc_flags)
{
    // 初始化 struct page，清除一些页面属性标记
    post_alloc_hook(page, order, gfp_flags);

     // 设置复合页
    if (order && (gfp_flags & __GFP_COMP))
```

```
        prep_compound_page(page, order);
    if (alloc_flags & ALLOC_NO_WATERMARKS)

        // 使用 set_page_XXX(page) 方法设置 page 的 PG_XXX 标志位
        set_page_pfmemalloc(page);
    Else
         // 使用 clear_page_XXX(page) 方法清除 page 的 PG_XXX 标志位
        clear_page_pfmemalloc(page);
}
```

由于现在拿到的 struct page 结构是刚刚从伙伴系统中申请出来的，里面可能包含一些无用的标记（上一次被使用过的，还没清理），所以需要将这些无用标记清理掉，并且在此基础上根据 gfp_flags 掩码对 struct page 进行初始化的准备工作。

```
inline void post_alloc_hook(struct page *page, unsigned int order,
                gfp_t gfp_flags)
{
    bool init = !want_init_on_free() && want_init_on_alloc(gfp_flags) &&
            !should_skip_init(gfp_flags);
    bool zero_tags = init && (gfp_flags & __GFP_ZEROTAGS);
    int i;

    set_page_private(page, 0);
    set_page_refcounted(page);
    arch_alloc_page(page, order);
    debug_pagealloc_map_pages(page, 1 << order);
    kernel_unpoison_pages(page, 1 << order);
    if (zero_tags) {
        for (i = 0; i != 1 << order; ++i)
            tag_clear_highpage(page + i);

        init = false;
    }
    if (!should_skip_kasan_unpoison(gfp_flags) &&
        kasan_unpoison_pages(page, order, init)) {
        if (kasan_has_integrated_init())
            init = false;
    } else {
        for (i = 0; i != 1 << order; ++i)
            page_kasan_tag_reset(page + i);
    }
    /* If memory is still not initialized, initialize it now. */
    if (init)
        kernel_init_pages(page, 1 << order);
    set_page_owner(page, order, gfp_flags);
```

```
    page_table_check_alloc(page, order);
}
```

set_page_private 将 struct page 里的 private 指针所指向的内容清空，private 指针在内核中的使用比较复杂，它会在不同场景下指向不同的内容。

set_page_refcounted 将页面的使用计数设置为 1 ，表示当前物理内存页正在被使用。

如果 gfp_flags 掩码中设置了 ___GFP_ZERO，这时就需要将这些 page 初始化为零页。

6.5 内存释放源码实现

内核释放物理内存的相关接口有：

```
void __free_pages(struct page *page, unsigned int order)
void free_pages(unsigned long addr, unsigned int order)
```

__free_pages ： 同 alloc_pages 函数对应，用于释放 2 的 order 次幂个内存页，释放的物理内存区域起始地址由该区域中的第一个 page 实例指针表示，也就是参数里的 struct page *page 指针。

```
void __free_pages(struct page *page, unsigned int order)
{
    //获取页面的 PageHead 标志（判断是否为复合页的页头
    int head = PageHead(page);
    //修改页面引用计数 / 状态
    if (put_page_testzero(page))
       //若引用计数归 0 ，就释放整个页块
        free_the_page(page, order);
    else if (!head)
       //页面不是复合页的头部
        while (order-- > 0)//遍历除第一页外的所有页面，逐个释放剩余页面
            free_the_page(page + (1 << order), order);
}
EXPORT_SYMBOL(__free_pages);
```

free_pages：同 __get_free_pages 函数对应，与 __free_pages 函数的区别是在释放物理内存时，使用了虚拟内存地址而不是 page 指针。

```
void free_pages(unsigned long addr, unsigned int order)
{
    if (addr != 0) {
       // 校验虚拟内存地址 addr 的有效性
        VM_BUG_ON(!virt_addr_valid((void *)addr));
       // 将虚拟内存地址 addr 转换为 page，最终还是调用 __free_pages
        __free_pages(virt_to_page((void *)addr), order);
    }
}
```

```
EXPORT_SYMBOL(free_pages);
```

内核也提供了 __free_page 和 free_page 两个宏，专门用于释放单个物理内存页。

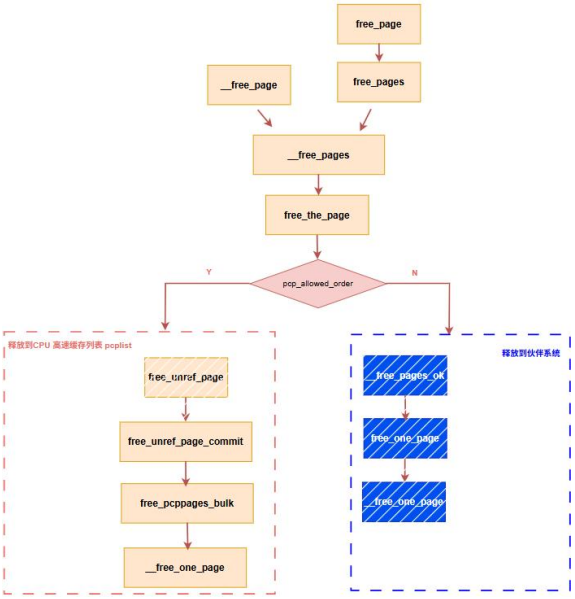
```
#define __free_page(page) __free_pages((page), 0)
#define free_page(addr) free_pages((addr), 0)
```

论是内核定义的这些用于释放内存的宏或是辅助函数，它们最终会调用 __free_pages，进而调用 free_the_page，这正是释放内存的核心逻辑。

```
static inline void free_the_page(struct page *page, unsigned int order)
{
    if (pcp_allowed_order(order))       /* Via pcp? */
        // 如果释放的内存页的阶数 order <= PAGE_ALLOC_COSTLY_ORDER，
        //则直接释放到 CPU 高速缓存列表 pcplist 中
      free_unref_page(page, order);
    else
        // 如果释放多页的话，则进入伙伴系统回收这部分内存
        __free_pages_ok(page, order, FPI_NONE);
}
```

伴系统回收内存的流程中最开始首先都会检查本次释放或者分配的是否只是一个物理内存页（order = 0），如果是则直接释放到 CPU 高速缓存列表 pcplist 中。如果不是则将内存释放回伙伴系统中。

内核释放物理内存的接口：__free_pages、free_pages、__free_page、free_page 的处理逻辑流程图：



6.5.1 将内存释放到 CPU 高速缓存列表 pcplist 中

free_unref_page 是将内存释放到 CPU 高速缓存列表 pcplist 中的入口。 其主要处理逻

辑是：将页面优先归还给当前 CPU 的 PCP 缓存（减少全局页分配器的锁竞争），若 PCP 缓存锁获取失败，则直接释放到全局页分配器。主要包含如下三个处理逻辑：

1. 如果页面迁移类型不是 UNMOVABLE、MOVABLE、RECLAIMABLE，并且是隔离类型（ISOLATE），即当前内存页被标记为禁止分配。直接调用 free_one_page 释放回伙伴系统。

2. 如果可以拿到当前 CPU 的 PCP 缓存自旋锁，调用 free_unref_page_commit 将页面释放到当前 CPU 的 pcplist 中，然后释放 PCP 锁。

3. 如果拿不到当前 CPU 的 PCP 缓存自旋锁，为避免死锁等待，直接调用 free_one_page 将页面释放到伙伴系统中。

```
/*
 * Free a pcp page
 */
void free_unref_page(struct page *page, unsigned int order)
{
    unsigned long __maybe_unused UP_flags;
    struct per_cpu_pages *pcp;
    //释放页面所属的内存域
    struct zone *zone;

     // 获取要释放的物理内存页对应的物理页号 pfn
    unsigned long pfn = page_to_pfn(page);

     // 关闭中断
    int migratetype, pcpmigratetype;

    if (!free_unref_page_prepare(page, pfn, order))
        return;
     // 内核这里只会将 UNMOVABLE,MOVABLE,RECLAIMABLE
    //这三种页面迁移类型放入 pcplist 中，其余的迁移类型均释放回伙伴系统
    migratetype = pcpmigratetype = get_pcppage_migratetype(page);
    if (unlikely(migratetype >= MIGRATE_PCPTYPES)) {
        if (unlikely(is_migrate_isolate(migratetype))) {
            //若是隔离类型（ISOLATE）：页面正在离线，
              //直接调用 free_one_page 释放回伙伴系统
            free_one_page(page_zone(page), page, pfn, order, migratetype,
FPI_NONE);
            return;
        }
         // 内核这里会将 HIGHATOMIC 和 CMA 类型页面按 MIGRATE_MOVABLE
        //类型处理处理。避免过度向伙伴分配器中释放页面
        pcpmigratetype = MIGRATE_MOVABLE;
    }
    zone = page_zone(page);
    pcp_trylock_prepare(UP_flags);
    //尝试获取当前 CPU 的 PCP 缓存自旋锁，
```

```
    //成功则返回 PCP 缓存指针，失败则返回 NULL。
    pcp = pcp_spin_trylock(zone->per_cpu_pageset);
    if (pcp) {
          // 成功获取 PCP 锁：调用 free_unref_page_commit
            //将页面加入当前 CPU 的 pcplist 中，然后释放 PCP 锁；
        free_unref_page_commit(zone, pcp, page, pcpmigratetype, order);
        pcp_spin_unlock(pcp);
    } else {
        //获取锁失败：直接调用 free_one_page 将页面释放到全局页分配器
        //避免死等锁，牺牲性能换并发.
        free_one_page(zone, page, pfn, order, migratetype, FPI_NONE);
    }
    pcp_trylock_finish(UP_flags);
}
```

需要注意的是，内核只会将 UNMOVABLE，MOVABLE，RECLAIMABLE 这三种页面迁移类型放入 CPU 高速缓存列表 pcplist 中，其余的迁移类型均需释放回伙伴系统。

```
enum migratetype {
    MIGRATE_UNMOVABLE,
    MIGRATE_MOVABLE,
    MIGRATE_RECLAIMABLE,
    MIGRATE_PCPTYPES,   /* the number of types on the pcp lists */
    MIGRATE_HIGHATOMIC = MIGRATE_PCPTYPES,
#ifdef CONFIG_CMA
    MIGRATE_CMA,
#endif
#ifdef CONFIG_MEMORY_ISOLATION
    MIGRATE_ISOLATE,    /* can't allocate from here */
#endif
    MIGRATE_TYPES
};
```

拿到 PCP 自选锁的情况下，free_unref_page 函数会调用 free_unref_page_commit ，将内存页释放至 CPU 高速缓存列表 pcplist 中，且在释放的过程中是不会响应中断。

将通过 this_cpu_ptr 获取的当前运行进程的 CPU 高速缓存列表 pcplist，传入到 free_unref_page_commit 中，然后在 free_unref_page_commit 函数中将要释放的物理内存页添加到对应迁移类型的 pcp->lists[migratetype]。在 CPU 高速缓存列表 per_cpu_pages 中，每个迁移类型对应一个 pcplist 。

如果当前 pcplist 中的页面数量 count 超过了规定的水位线 high 的值，说明现在 pcplist 中的页面太多了，需要从 pcplist 中释放 batch 个物理页面到伙伴系统中。这个过程称之为惰性合并。

free_unref_page_commit 函数具体实现如下：

```
static void free_unref_page_commit(struct zone *zone, struct per_cpu_pages
*pcp,
```

```
                    struct page *page, int migratetype,
                    unsigned int order)
{
    int high, batch;
    //pcp_list 的索引
    int pindex;
    bool free_high = false;
    //alloc_factor 是 PCP 的分配因子（控制批量分配的页面数）
    //分配时增加因子、释放时减少因子，避免 PCP 缓存过度分配。
    pcp->alloc_factor >>= 1;
    __count_vm_events(PGFREE, 1 << order);
    pindex = order_to_pindex(migratetype, order);
     // 将要释放的物理内存页添加到 pcplist 中
    list_add(&page->pcp_list, &pcp->lists[pindex]);
    //更新 PCP 缓存中的物理页数
    pcp->count += 1 << order;
    //获取 batch 的值
    batch = READ_ONCE(pcp->batch);
    //释放页的阶数 order 非零并且 order <= PAGE_ALLOC_COSTLY_ORDER 时
    if (order && order <= PAGE_ALLOC_COSTLY_ORDER) {
        free_high = (pcp->free_count >= batch &&
                 (pcp->flags & PCPF_PREV_FREE_HIGH_ORDER) &&
                 (!(pcp->flags & PCPF_FREE_HIGH_BATCH) ||
                  pcp->count >= READ_ONCE(batch)));
        pcp->flags |= PCPF_PREV_FREE_HIGH_ORDER;
    } else if (pcp->flags & PCPF_PREV_FREE_HIGH_ORDER) {
        pcp->flags &= ~PCPF_PREV_FREE_HIGH_ORDER;
    }
    if (pcp->free_count < (batch << CONFIG_PCP_BATCH_SCALE_MAX))
    //若 free_count 未超过上限，则累加本次释放的物理页数
        pcp->free_count += (1 << order);


      //动态计算 PCP 缓存的高水位：结合 PCP 配置、内存域状态、
      //批量大小、高阶释放标记，得到当前允许的 PCP 缓存最大页面数
    high = nr_pcp_high(pcp, zone, batch, free_high);
    if (pcp->count >= high) {
    // 将 PCP 缓存中指定数量的页面批量释放到伙伴系统
    //其中，nr_pcp_free 计算本次要释放的页面数
        free_pcppages_bulk(zone, nr_pcp_free(pcp, batch, high, free_high),
                    pcp, pindex);
        if (test_bit(ZONE_BELOW_HIGH, &zone->flags) &&
            zone_watermark_ok(zone, 0, high_wmark_pages(zone),
                      ZONE_MOVABLE, 0))
```

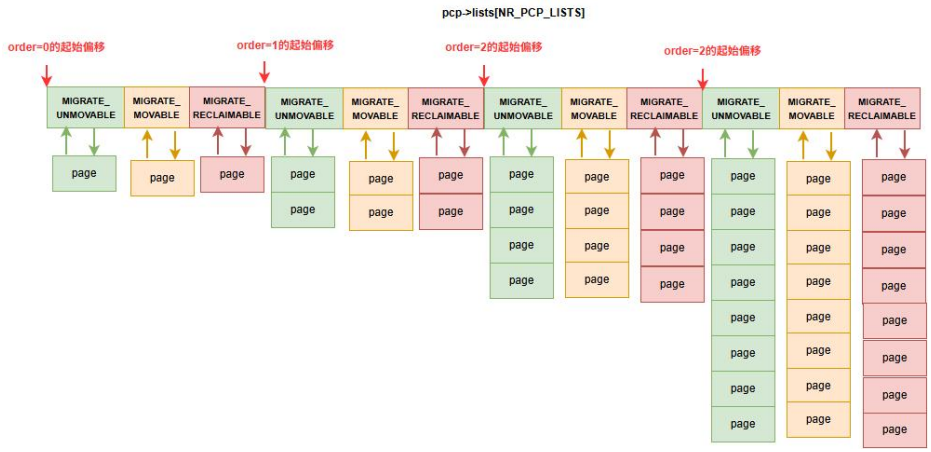
```
            //若水位线恢复到 WMARK_HIGH，清除 ZONE_BELOW_HIGH 标志位
             clear_bit(ZONE_BELOW_HIGH, &zone->flags);
    }
}
```

其中，order_to_pindex 函数将 PCP 缓存的“页面阶数（order）+ 迁移类型（migratetype）”二维属性，转换为 PCP 链表数组（lists[NR_PCP_LISTS]）一维索引。

```
static inline unsigned int order_to_pindex(int migratetype, int order)
{
#ifdef CONFIG_TRANSPARENT_HUGEPAGE
    //高阶页面处理
    if (order > PAGE_ALLOC_COSTLY_ORDER) {
        VM_BUG_ON(order != pageblock_order);
        //是低阶索引的 上限，
        //之后开始都是高阶页(order > PAGE_ALLOC_COSTLY_ORDER)的索引
        return NR_LOWORDER_PCP_LISTS;
    }
#else
    VM_BUG_ON(order > PAGE_ALLOC_COSTLY_ORDER);
#endif
    /*把 PCP 链表数组（lists[NR_PCP_LISTS]）按 “阶数” 划分为多个连续的块，
    每个块的大小等于 MIGRATE_PCPTYPES（3），对应同一阶数下的 3 种迁移类型；
    最终索引 = 阶数块的起始偏移 + 迁移类型在块内的偏移。如下图，所示*/
    //流程走到此处时，满足 0 ≤ order ≤ PAGE_ALLOC_COSTLY_ORDER
    return (MIGRATE_PCPTYPES * order) + migratetype;
}
```

```
#define PAGE_ALLOC_COSTLY_ORDER 3


#define NR_LOWORDER_PCP_LISTS (MIGRATE_PCPTYPES * (PAGE_ALLOC_COSTLY_ORDER
+ 1))
```

PCP 链表数组（lists[NR_PCP_LISTS]）示意图：



第一步：计算阶数块的起始偏移（MIGRATE_PCPTYPES * order）每个阶数（order）占用 3 个连续的数组位置（对应 3 种迁移类型），如上图所示

order=0：起始偏移 = 3×0 = 0；

order=1：起始偏移 = 3×1 = 3；

order=2：起始偏移 = 3×2 = 6；

order=3：起始偏移 = 3×3 = 9；

（PAGE_ALLOC_COSTLY_ORDER 的值为 3）

第二步：叠加迁移类型偏移（+ migratetype）在当前阶数块内，按迁移类型分配具体位置：

migratetype=0（不可移动）：偏移 0；

migratetype=1（可回收）：偏移 1；

migratetype=2（可移动）：偏移 2。

6.5.2 将内存释放到伙伴系统中

释放的内存页的阶数 order > 3 时，内核会将这些内存页回收至伙伴系统中，释放内存时伙伴系统的入口函数为 __free_pages_ok：

```
static void __free_pages_ok(struct page *page, unsigned int order,
                fpi_t fpi_flags)
{
    int migratetype;

      // 获取释放内存页对应的物理页号 pfn
    unsigned long pfn = page_to_pfn(page);
    struct zone *zone = page_zone(page);

    // 在将内存页回收至伙伴系统之前，需要将内存页 page 相关的无用属性清理一下
    if (!free_pages_prepare(page, order, fpi_flags))
        return;
```

```
    // 获取页面迁移类型，后续会将内存页释放
    //至伙伴系统中的 free_list[migratetype] 中
    migratetype = get_pfnblock_migratetype(page, pfn);

    //进入伙伴系统，释放内存
    free_one_page(zone, page, pfn, order, migratetype, fpi_flags);
    __count_vm_events(PGFREE, 1 << order);
}
```

__free_pages_ok 函数的核心逻辑是：在将内存页回收至伙伴系统之前，需要将这些内存页的 page 结构清理一下，将无用的属性置空。然后调用 free_one_page 将清理之后干净的 page 结构回收至伙伴系统中。

6.5.3 free_one_page 函数源码分析

将处理后页面放到伙伴系统的逻辑主要包含在 free_one_page 函数中，free_one_page 主要包含 5 部分逻辑：
第 1 部分逻辑：调用 find_buddy_page_pfn 找到当前页的伙伴页。
第 2 部分逻辑：调用 find_buddy_page_pfn 找到当前页的伙伴页。高阶页块迁移类型校验。
第 3 部分逻辑：处理伙伴页。
第 4 部分逻辑：页块合并。
第 5 部分逻辑：合并完成：收尾处理。
其中，1~4 是一个循环不断合并空闲页块的过程。
具体源码如下：

```
static void free_one_page(struct zone *zone,
                struct page *page, unsigned long pfn,
                unsigned int order,
                int migratetype, fpi_t fpi_flags)
{
    unsigned long flags;

    spin_lock_irqsave(&zone->lock, flags);
    if (unlikely(has_isolate_pageblock(zone) ||
        is_migrate_isolate(migratetype))) {
        migratetype = get_pfnblock_migratetype(page, pfn);
    }

    // 正式进入伙伴系统回收内存
    __free_one_page(page, pfn, zone, order, migratetype, fpi_flags);
    spin_unlock_irqrestore(&zone->lock, flags);
}
```

free_one_page 调用__free_one_page 函数处理释放逻辑，__free_one_page 源码如下：

```
    /*
    Freeing function for a buddy system allocator.
```

```
    */
static inline void __free_one_page(struct page *page,
        unsigned long pfn,
        struct zone *zone, unsigned int order,
        int migratetype, fpi_t fpi_flags)
{
    struct capture_control *capc = task_capc(zone);

    // 伙伴页的 PFN
    unsigned long buddy_pfn = 0;
    // 合并后页块的起始 PFN
    unsigned long combined_pfn;
    // 伙伴内存块的首页 page 指针
    struct page *buddy;
    bool to_tail;

    VM_BUG_ON(!zone_is_initialized(zone));
    VM_BUG_ON_PAGE(page->flags & PAGE_FLAGS_CHECK_AT_PREP, page);
    VM_BUG_ON(migratetype == -1);
    if (likely(!is_migrate_isolate(migratetype)))
       //带释放的页非 MIGRATE_ISOLATE 类型，
        //在统计信息中更新内存域 zone 的总的空闲页数。
        __mod_zone_freepage_state(zone, 1 << order, migratetype);
    VM_BUG_ON_PAGE(pfn & ((1 << order) - 1), page);
    VM_BUG_ON_PAGE(bad_range(zone, page), page);
    while (order < MAX_PAGE_ORDER) {
        if (compaction_capture(capc, page, order, migratetype)) {
            __mod_zone_freepage_state(zone, -(1 << order),
                                migratetype);
            return;
        }

         // 在 free_area[order] 中查找伙伴内存块 首页的指针
        buddy = find_buddy_page_pfn(page, pfn, order, &buddy_pfn);
        if (!buddy)
            goto done_merging;// 无伙伴页，终止合并
       //高阶页块（≥pageblock_order）迁移类型校验
        if (unlikely(order >= pageblock_order)) {
            int buddy_mt = get_pfnblock_migratetype(buddy, buddy_pfn);
            if (migratetype != buddy_mt
                    && (!migratetype_is_mergeable(migratetype) ||
                        !migratetype_is_mergeable(buddy_mt)))
                goto done_merging;// 无伙伴页，终止合并
        }
```

```
        if (page_is_guard(buddy))
            //如果是调试保护页，清除调试保护标志位
            clear_page_guard(zone, buddy, order, migratetype);
        else
            // 从空闲链表删除伙伴页
            del_page_from_free_list(buddy, zone, order);
        // 合并后新内存块首页 page 的 pfn
        combined_pfn = buddy_pfn & pfn;

        // 合并后新内存块首页 page 指针
        page = page + (combined_pfn - pfn);

        // 以合并后的新内存块为基础继续向高阶 free_area 合并
        pfn = combined_pfn;
        order++;
    }
done_merging:
    // 表示在当前伙伴系统 free_area[order] 中没有找到伙伴内存块，停止合并
    // 设置内存块的分配阶 order，存储在第一个 page 结构中的 private 属性中
    set_buddy_order(page, order);

    //决定将页块加入空闲链表的头部/尾部
    if (fpi_flags & FPI_TO_TAIL)
        to_tail = true;  // 强制加入尾部
    else if (is_shuffle_order(order))
        // 随机选择（内存洗牌，降低碎片）
        to_tail = shuffle_pick_tail();
    Else
         // 计算前页块是否易与更高阶伙伴页合并，再决定是否插入尾部
        to_tail = buddy_merge_likely(pfn, buddy_pfn, page, order);
    if (to_tail)
         //加入空闲链表尾部
        add_to_free_list_tail(page, zone, order, migratetype);
    else
        //加入空闲链表头部
        add_to_free_list(page, zone, order, migratetype);
    /* Notify page reporting subsystem of freed page */
    if (!(fpi_flags & FPI_SKIP_REPORT_NOTIFY))
        page_reporting_notify_free(order);
}
```

第 1 部分逻辑：调用 find_buddy_page_pfn 找到当前页的伙伴页。

find_buddy_page_pfn。主要处理逻辑为：

1) 基于待释放页的 PFN 和阶数，计算其伙伴页的 PFN。函数：__find_buddy_pfn

2) 通过指针偏移快速定位伙伴页的 struct page 结构体
3) 校验伙伴页的合法性（是否符合伙伴系统规则）。函数：page_is_buddy
4) 返回合法的伙伴页指针（或 NULL），同时通过输出参数返回伙伴页 PFN

```
static inline struct page *find_buddy_page_pfn(struct page *page,
            unsigned long pfn, unsigned int order, unsigned long *buddy_pfn)
{
    //计算伙伴页的 PFN
    unsigned long __buddy_pfn = __find_buddy_pfn(pfn, order);
    struct page *buddy;


    //根据伙伴页的 PFN、待释放页的 PFN、待释放页的指针
    //定位伙伴页的 struct page 指针。（伙伴页之间物理内存是连续的）
    buddy = page + (__buddy_pfn - pfn);
    if (buddy_pfn)
        *buddy_pfn = __buddy_pfn;
    //校验伙伴页合法性
    if (page_is_buddy(page, buddy, order))
        return buddy;
    return NULL;
}
```

其中，计算伙伴页的 PFN 的函数__find_buddy_pfn 具体逻辑为：

根据当前释放内存块的 pfn，以及当前释放内存块的分配阶 order 来确定其伙伴内存块的 pfn。源码如下：

```
static inline unsigned long
__find_buddy_pfn(unsigned long page_pfn, unsigned int order)
{
    return page_pfn ^ (1 << order);
}
```

首先通过 1 << order 左移操作确定要查找伙伴内存块的分配阶，因为伙伴关系最重要的一点就是它们必须是大小相等的两个内存块。然后通过与要释放内存块的 pfn 进行异或操作就得到了伙伴内存块的 pfn 。

校验伙伴页合法性的函数 page_is_buddy 具体实现为：

```
static inline bool page_is_buddy(struct page *page, struct page *buddy,
                unsigned int order)
{
    if (!page_is_guard(buddy) && !PageBuddy(buddy))
        return false;

    if (buddy_order(buddy) != order)
        return false;
    if (page_zone_id(page) != page_zone_id(buddy))
```

```
        return false;
    VM_BUG_ON_PAGE(page_count(buddy) != 0, buddy);
    return true;
}
```

从函数 page_is_buddy 可以看出，同时满足下述四点的两个内存块即为伙伴关系:

1) 伙伴系统所管理的内存页必须是可用的，不能处于内存空洞中，通过 page_is_guard 函数判断。

2) 伙伴必须是空闲的内存块，这些内存块必须存在于伙伴系统中，组成内存块的内存页 page 结构中的 flag 标志设置了 PG_buddy 标记。通过 PageBuddy 判断这些内存页是否在伙伴系统中。

3) 两个互为伙伴的内存块必须拥有相同的分配阶 order，也就是它们之间的大小尺寸必须一致。通过 page_order(buddy) == order 判断

4) 互为伙伴关系的内存块必须处于相同的物理内存区域 zone 中。通过 page_zone_id(page) == page_zone_id(buddy) 判断。

第 2 部分逻辑：调用 find_buddy_page_pfn 找到当前页的伙伴页。高阶页块迁移类型校验。高阶页块合并需严格匹配迁移类型，防止 CMA / 隔离 / HIGHATOMIC 页与普通页合并。

```
        if (unlikely(order >= pageblock_order)) {
            int buddy_mt = get_pfnblock_migratetype(buddy, buddy_pfn);

            if (migratetype != buddy_mt
                    && (!migratetype_is_mergeable(migratetype) ||
                        !migratetype_is_mergeable(buddy_mt)))
                goto done_merging;
        }
```

第 3 部分逻辑：处理伙伴页。若伙伴页是保护页 ，清除保护标志（恢复为普通页）；否则将待释放页直接从链表删除。

```
        if (page_is_guard(buddy))
            clear_page_guard(zone, buddy, order, migratetype);
        else
            del_page_from_free_list(buddy, zone, order);
        combined_pfn = buddy_pfn & pfn;
```

第 4 部分逻辑：页块合并。

通过 combined_pfn = buddy_pfn & pfn 计算出合并后的起始 PFN；

通过 page = page + (combined_pfn - pfn) 计算出合并后的头部页的指针；

通过 pfn = combined_pfn 将待释放页的 PFN 更新为合并后页块中首页的 PFN；因为此时待释放页已经合并到伙伴内存块中，需要将这个伙伴继续往高阶合并。因此，待合并页的 PFN 就这个伙伴内存块的 PFN（combined_pfn）。如下图所示，page0 是待释放页，其页帧号是 pfn，它的伙伴页是 page1，其帧号是 buddy_pfn，两者合并后成为新的高阶内存块，其页帧号是 combined_pfn。在继续寻找 这个 新的高阶内存块的伙伴块之前，需要将 pfn 更新为 buddy_pfn。表示此时，待释放的内存块是 这个 新的高阶内存块。

order++ 阶数+1，继续尝试更高阶合并。

```
        combined_pfn = buddy_pfn & pfn;
        page = page + (combined_pfn - pfn);
        pfn = combined_pfn;
        order++;
```

第 5 部分逻辑：合并完成：收尾处理。

标记页块阶数

决定链表插入位置

将空闲页块加入到对应的空闲链表中

```
done_merging:
    // 表示在当前伙伴系统 free_area[order] 中没有找到伙伴内存块，停止合并
    // 设置内存块的分配阶 order，存储在第一个 page 结构中的 private 属性中
    set_buddy_order(page, order);

    //决定将页块加入空闲链表的头部/尾部
    if (fpi_flags & FPI_TO_TAIL)
        to_tail = true;  // 强制加入尾部
    else if (is_shuffle_order(order))
        // 随机选择（内存洗牌，降低碎片）
        to_tail = shuffle_pick_tail();
    Else
         // 计算前页块是否易与更高阶伙伴页合并，再决定是否插入尾部
        to_tail = buddy_merge_likely(pfn, buddy_pfn, page, order);
    if (to_tail)
         //加入空闲链表尾部
        add_to_free_list_tail(page, zone, order, migratetype);
    else
        //加入空闲链表头部
        add_to_free_list(page, zone, order, migratetype);
```